

# Group 13

Group Members:

- Diego Arevalo Fernandez
- Iain Bustillos
- Christina Cho
- John Shan

## Research Questions

1. How do air quality patterns, as measured through the mean Air Quality Index scores, differ between coastal and inland regions of San Diego county, with coastal regions being measured as the zip codes that contain the coast and inland regions being measured as zip codes that do not contain the coast?
2. Is there a correlation between the air quality patterns, as measured through the mean Air Quality Index scores, and disparities in health outcomes, as measured by rates of asthma in each zip code, when controlling for other variables (income, median age, etc.)?

## Data Collection and Exploration

```
In [1]: # Import necessary packages
        #%pip install pgeocode
        import pandas as pd
        import numpy as np
        import matplotlib.pyplot as plt
        from matplotlib.colors import ListedColormap, BoundaryNorm
        import seaborn as sns
        import scipy.stats as stats
        import pingouin as pg
        import pgeocode
        import numpy as np
        import statsmodels.formula.api as smf
        import statsmodels.api as sm
        import folium
        from folium.features import DivIcon
```

First, in order to answer our research questions, we need data on the Air Quality Index scores for each region of San Diego.

## Wrangling AQI Data

```
In [2]: file_path = 'aqi_data.csv'
aqi_df = pd.read_csv(file_path)
aqi_df.head()
```

```
Out[2]:
```

|   | Date       | Source | Site ID  | POC | Daily Max 8-hour Ozone Concentration | Units | Daily AQI Value | Local Site Name | Daily Obs Count | Perc Compl |
|---|------------|--------|----------|-----|--------------------------------------|-------|-----------------|-----------------|-----------------|------------|
| 0 | 01/01/2021 | AQS    | 60730001 | 1   | 0.040                                | ppm   | 37              | Chula Vista     | 17              | 100        |
| 1 | 01/02/2021 | AQS    | 60730001 | 1   | 0.038                                | ppm   | 35              | Chula Vista     | 17              | 100        |
| 2 | 01/03/2021 | AQS    | 60730001 | 1   | 0.041                                | ppm   | 38              | Chula Vista     | 17              | 100        |
| 3 | 01/04/2021 | AQS    | 60730001 | 1   | 0.039                                | ppm   | 36              | Chula Vista     | 17              | 100        |
| 4 | 01/05/2021 | AQS    | 60730001 | 1   | 0.034                                | ppm   | 31              | Chula Vista     | 17              | 100        |

5 rows × 21 columns



```
In [3]: aqi_df.columns
```

```
Out[3]: Index(['Date', 'Source', 'Site ID', 'POC',
              'Daily Max 8-hour Ozone Concentration', 'Units', 'Daily AQI Value',
              'Local Site Name', 'Daily Obs Count', 'Percent Complete',
              'AQS Parameter Code', 'AQS Parameter Description', 'Method Code',
              'CBSA Code', 'CBSA Name', 'State FIPS Code', 'State',
              'County FIPS Code', 'County', 'Site Latitude', 'Site Longitude'],
              dtype='object')
```

Let's visualize the stations' locations and their average AQI score.

```
In [4]: grouped = (
    aqi_df.groupby("Local Site Name", as_index=False)
    .agg({
        "Daily AQI Value": "mean",
        "Site Latitude": "first",
        "Site Longitude": "first"
```

```

    })
)
grouped.rename(columns={"Daily AQI Value": "avg_aqi"}, inplace=True)

center_lat = grouped["Site Latitude"].mean()
center_lon = grouped["Site Longitude"].mean()

m = folium.Map(
    location=[center_lat, center_lon],
    zoom_start=9,
    tiles="OpenStreetMap"
)

for _, row in grouped.iterrows():
    val = row["avg_aqi"]

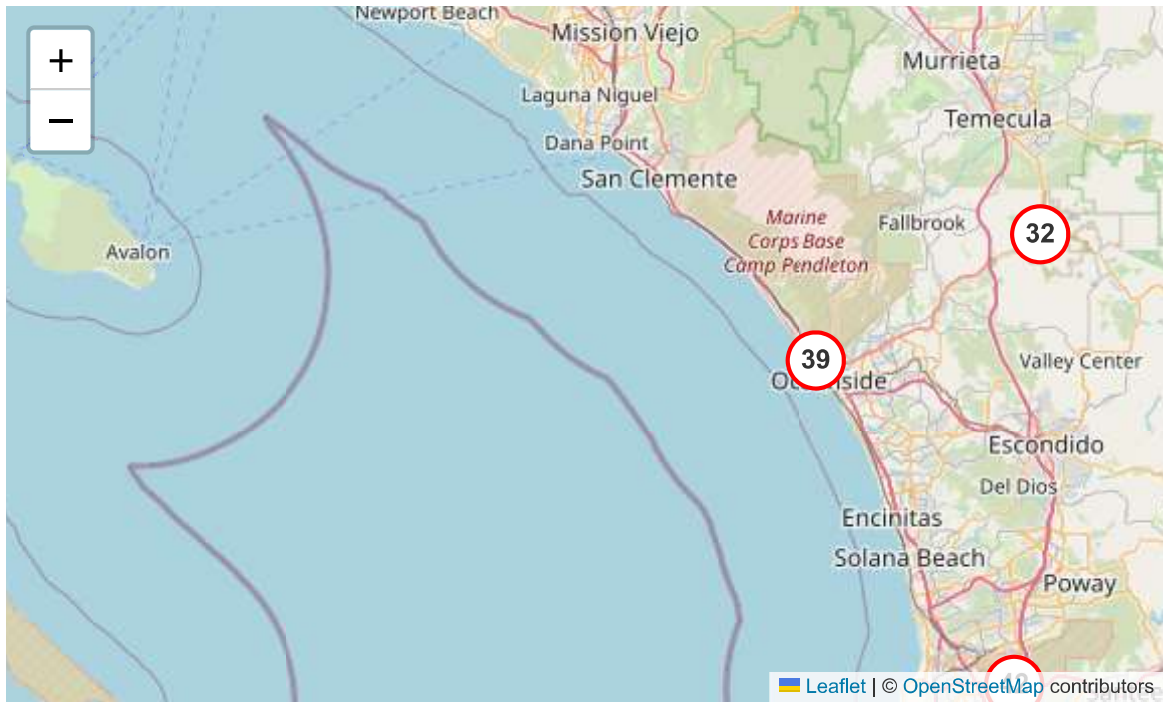
    # draw a circle for visual area
    folium.Circle(
        location=[row["Site Latitude"], row["Site Longitude"]],
        radius=2000,          # adjust radius (in meters)
        color="red",
        fill=True,
        fill_opacity=0.4
    ).add_to(m)

    # put AQI number on top of the circle
    folium.Marker(
        location=[row["Site Latitude"], row["Site Longitude"]],
        icon=DivIcon(
            icon_size=(30, 30),
            icon_anchor=(15, 15),
            html=f"""
                <div style="
                    background-color: white;
                    border-radius: 50%;
                    border: 2px solid red;
                    width: 30px;
                    height: 30px;
                    text-align: center;
                    line-height: 26px;
                    font-size: 10pt;
                    font-weight: bold;
                ">
                    {int(round(val))}
                </div>
            """
        ),
        tooltip=f"{row['Local Site Name']} - Avg AQI: {val:.1f}"
    ).add_to(m)

m

```

Out[4]:



Now, let's simplify the dataset by only selecting the columns relevant for our data analysis, and also change date into a pd DateTime object.

```
In [5]: aqi_df = aqi_df[['Date', 'Site ID', 'Daily AQI Value', 'Local Site Name', 'Daily Ob
aqi_df['Date'] = pd.to_datetime(aqi_df['Date'])
```

Next, let us add a column to label the sites as being coastal or inland.

```
In [6]: coastal_stations = [
    "Chula Vista",
    "Camp Pendleton",
    "San Diego - Sherman Elementary School"
]

urban_stations = [
    "San Diego - Kearny Villa Rd.",
    "El Cajon - Lexington Elementary School",
    "Donovan"
]

inland_stations = [
    "Alpine",
    "Pala Airpad",
    "La Posta Band of Indians"
]

aqi_df['Coastal'] = np.where(aqi_df['Local Site Name'].isin(coastal_stations), 1, 0)
aqi_df['Inland'] = np.where(aqi_df['Local Site Name'].isin(inland_stations), 1, 0)
aqi_df.head()
```



Out[6]:

|   | Date       | Site ID  | Daily<br>AQI<br>Value | Local<br>Site<br>Name | Daily<br>Obs<br>Count | Site<br>Latitude | Site<br>Longitude | Coastal | Inland |
|---|------------|----------|-----------------------|-----------------------|-----------------------|------------------|-------------------|---------|--------|
| 0 | 2021-01-01 | 60730001 | 37                    | Chula Vista           | 17                    | 32.631242        | -117.059088       | 1       | 0      |
| 1 | 2021-01-02 | 60730001 | 35                    | Chula Vista           | 17                    | 32.631242        | -117.059088       | 1       | 0      |
| 2 | 2021-01-03 | 60730001 | 38                    | Chula Vista           | 17                    | 32.631242        | -117.059088       | 1       | 0      |
| 3 | 2021-01-04 | 60730001 | 36                    | Chula Vista           | 17                    | 32.631242        | -117.059088       | 1       | 0      |
| 4 | 2021-01-05 | 60730001 | 31                    | Chula Vista           | 17                    | 32.631242        | -117.059088       | 1       | 0      |

## EDA for AQI Data

In [7]: `aqi_df.describe()`

Out[7]:

|       | Date                             | Site ID      | Daily AQI<br>Value | Daily Obs<br>Count | Site Latitude | Long    |
|-------|----------------------------------|--------------|--------------------|--------------------|---------------|---------|
| count | 14489                            | 1.448900e+04 | 14489.000000       | 14489.000000       | 14489.000000  | 14489.0 |
| mean  | 2023-06-25<br>12:29:39.998619392 | 6.073153e+07 | 42.953965          | 17.634343          | 32.860506     | -117.0  |
| min   | 2021-01-01<br>00:00:00           | 6.073000e+07 | 0.000000           | 13.000000          | 32.578165     | -117.3  |
| 25%   | 2022-04-17<br>00:00:00           | 6.073101e+07 | 35.000000          | 17.000000          | 32.710177     | -117.1  |
| 50%   | 2023-07-08<br>00:00:00           | 6.073102e+07 | 41.000000          | 17.000000          | 32.789565     | -117.0  |
| 75%   | 2024-08-30<br>00:00:00           | 6.073103e+07 | 46.000000          | 17.000000          | 32.845709     | -116.9  |
| max   | 2025-12-09<br>00:00:00           | 6.073999e+07 | 161.000000         | 24.000000          | 33.362593     | -116.3  |
| std   | NaN                              | 2.288300e+03 | 15.529692          | 2.651911           | 0.251168      | 0.2     |



In [8]: `stations_per_day = aqi_df.groupby('Date')['Local Site Name'].nunique()  
stations_per_day.describe()`

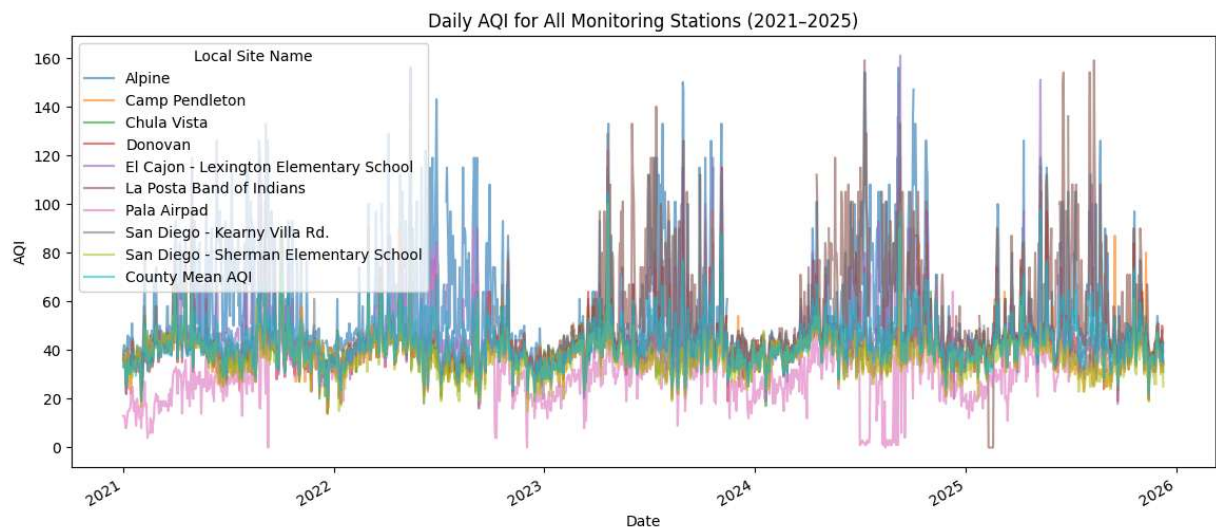
```
Out[8]: count    1800.000000
        mean      8.049444
        std       0.850269
        min       4.000000
        25%       8.000000
        50%       8.000000
        75%       9.000000
        max       9.000000
        Name: Local Site Name, dtype: float64
```

```
In [9]: pivot = aqi_df.pivot_table(
        index='Date',
        columns='Local Site Name',
        values='Daily AQI Value'
    )
```

```
In [10]: pivot['County Mean AQI'] = pivot.mean(axis=1)
```

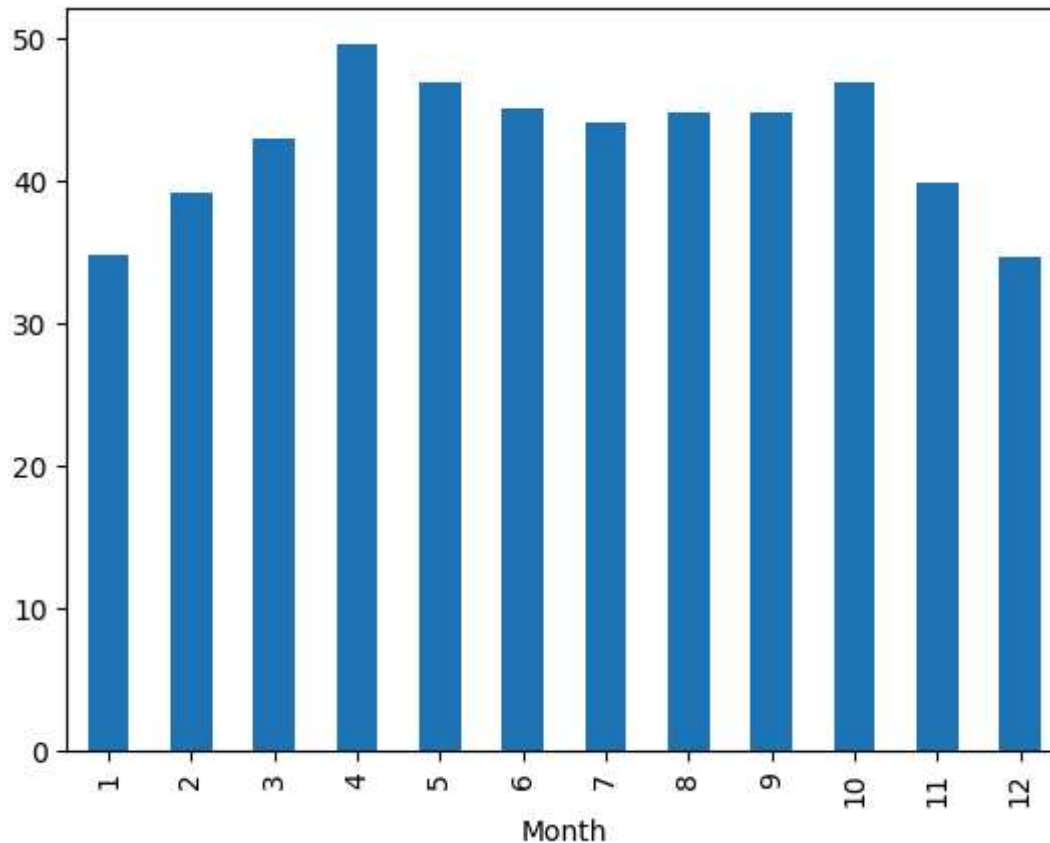
```
In [11]: pivot.plot(figsize=(14,6), alpha=0.6)
        plt.title("Daily AQI for All Monitoring Stations (2021-2025)")
        plt.ylabel("AQI")
```

```
Out[11]: Text(0, 0.5, 'AQI')
```



```
In [12]: ## Mean AQI by Month
        pivot['Month'] = pivot.index.month
        pivot.groupby('Month')['County Mean AQI'].mean().plot(kind='bar')
```

```
Out[12]: <Axes: xlabel='Month'>
```



```
In [13]: pivot['7d'] = pivot['County Mean AQI'].rolling(7).mean()
pivot['30d'] = pivot['County Mean AQI'].rolling(30).mean()

heavy_days = pivot[pivot['County Mean AQI'] > 100]
```

```
In [14]: coastal = pivot[coastal_stations].mean(axis=1)
inland = pivot[inland_stations].mean(axis=1)

coastal_smooth = coastal.rolling(window=7, center=True).mean()
inland_smooth = inland.rolling(window=7, center=True).mean()

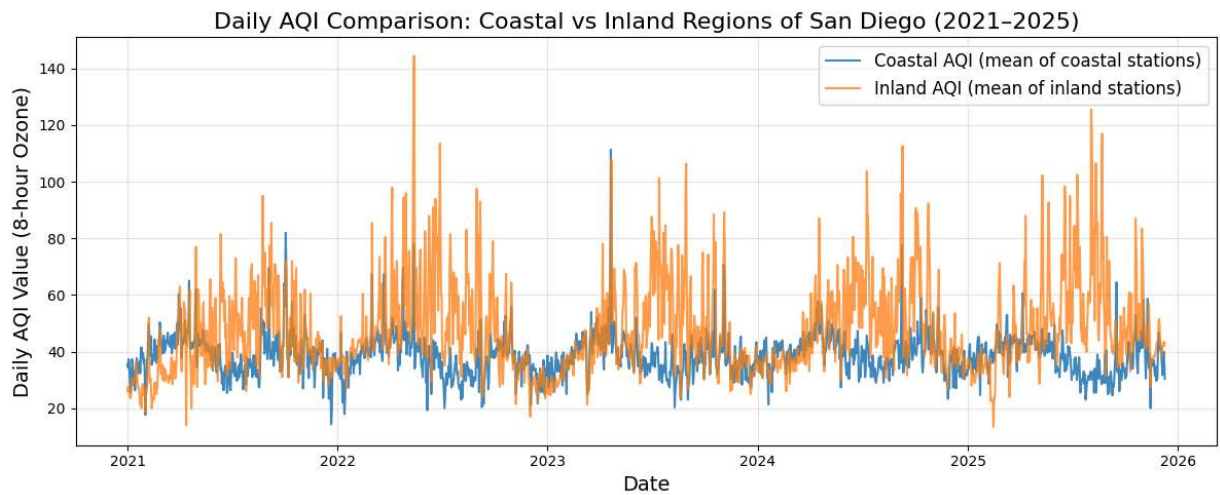
plt.figure(figsize=(12,5))

plt.plot(coastal, label="Coastal AQI (mean of coastal stations)", alpha=0.85)
plt.plot(inland, label="Inland AQI (mean of inland stations)", alpha=0.75)

plt.title("Daily AQI Comparison: Coastal vs Inland Regions of San Diego (2021-2025)")
plt.xlabel("Date", fontsize=14)
plt.ylabel("Daily AQI Value (8-hour Ozone)", fontsize=14)

plt.legend(fontsize=12)
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()
```



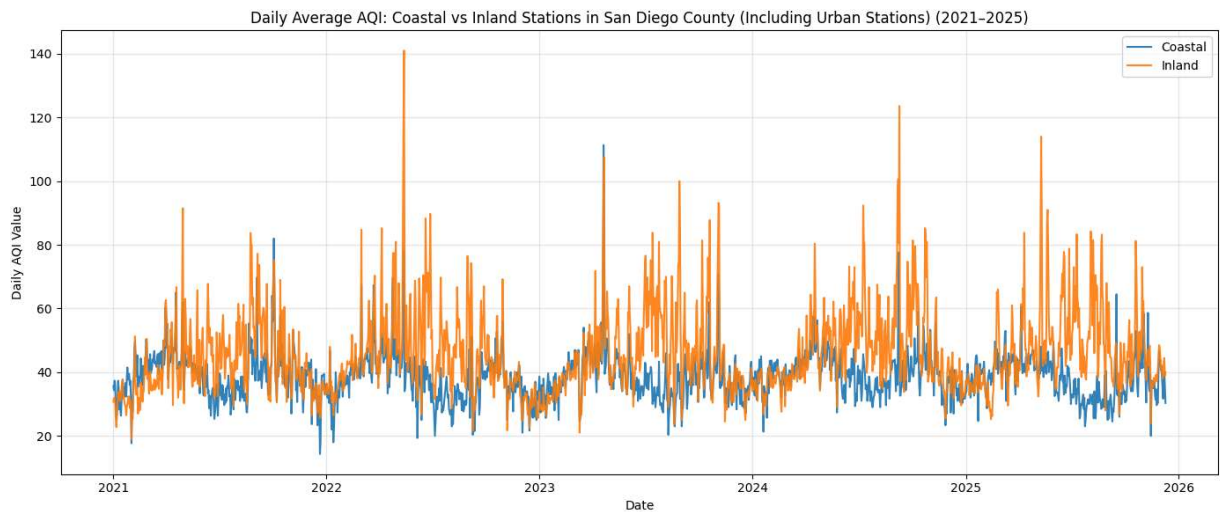
```
In [15]: ## Coastal vs Inland (including urban)
coastal_urban_stations = [
    'Chula Vista',
    'Camp Pendleton',
    'San Diego - Sherman Elementary School'
]

inland_urban_stations = [
    'Alpine',
    'Pala Airpad',
    'La Posta Band of Indians',
    'El Cajon - Lexington Elementary School',
    'Donovan'
]
```

```
coastal_urban = pivot[coastal_urban_stations].mean(axis=1)
inland_urban = pivot[inland_urban_stations].mean(axis=1)
```

```
In [16]: plt.figure(figsize=(14,6))
plt.plot(coastal_urban.index, coastal_urban, label="Coastal", alpha=0.9)
plt.plot(inland_urban.index, inland_urban, label="Inland", alpha=0.9)

plt.title("Daily Average AQI: Coastal vs Inland Stations in San Diego County (Inclu
plt.xlabel("Date")
plt.ylabel("Daily AQI Value")
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



```
In [17]: pivot.isna().sum().sort_values().drop(['County Mean AQI', 'Month', '7d', '30d'])
```

```
Out[17]: Local Site Name
San Diego - Sherman Elementary School    34
Alpine                                   39
San Diego - Kearny Villa Rd.             41
Donovan                                  53
Camp Pendleton                           119
Pala Airpad                              165
El Cajon - Lexington Elementary School  191
Chula Vista                              237
La Posta Band of Indians                 832
dtype: int64
```

## Obtaining and Cleaning Demographic Data

```
In [18]: ## Loading packages
#pip install census requests us
import os
import pandas as pd
import numpy as np
from census import Census
from us import states
import requests
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
import seaborn as sns
import matplotlib.pyplot as plt
```

```
In [19]: # Census API key
plt.rcParams["figure.figsize"] = (8, 4)

# --- keys & constants ---
CENSUS_API_KEY = "ce8b9610a02a2db4fd9e5ccb0412895621293667"
c = Census(CENSUS_API_KEY)
```

```
In [20]: STATE_FIPS = states.CA.fips          # '06'
        SAN_DIEGO_COUNTY_FIPS = "073"      # San Diego County
        START_YEAR = 2018
        END_YEAR = 2023

        # folder for any local CSVs you download
        DATA_DIR = "data"
        os.makedirs(DATA_DIR, exist_ok=True)
```

```
In [21]: ## Get Census data by zipcode

        ACS_VARS = {
            "B01003_001E": "pop_total",
            "B19013_001E": "med_income",
            "B01002_001E": "med_age",
            "B02001_002E": "white_alone",
            "B02001_003E": "black_alone",
            "B02001_005E": "asian_alone",
            "B03002_012E": "hispanic_any",
            "B17001_002E": "pop_below_poverty",
        }

        acs_cols = tuple(ACS_VARS.keys()) + ("NAME",)

        STATE_FIPS = states.CA.fips # "06"

        # Pull ALL ZCTAs in California
        zcta_raw = c.acs5.state_zipcode(
            acs_cols,
            STATE_FIPS,
            Census.ALL
        )

        acs_zip = pd.DataFrame(zcta_raw).rename(columns=ACS_VARS)

        # Rename + format ZIP/ZCTA key
        acs_zip.rename(columns={"zip code tabulation area": "Zip_Code"}, inplace=True)
        acs_zip["Zip_Code"] = acs_zip["Zip_Code"].astype(str).str.zfill(5)

        # Make sure numeric cols are numeric (API often returns strings)
        num_cols = list(ACS_VARS.values())
        acs_zip[num_cols] = acs_zip[num_cols].apply(pd.to_numeric, errors="coerce")

        # Fractions
        acs_zip["p_white"] = acs_zip["white_alone"] / acs_zip["pop_total"]
        acs_zip["p_black"] = acs_zip["black_alone"] / acs_zip["pop_total"]
        acs_zip["p_asian"] = acs_zip["asian_alone"] / acs_zip["pop_total"]
        acs_zip["p_hispanic"] = acs_zip["hispanic_any"] / acs_zip["pop_total"]
        acs_zip["p_poverty"] = acs_zip["pop_below_poverty"] / acs_zip["pop_total"]
```

```
In [22]: ## Reduce Census Dataset to Zip Codes Associated With San Diego County
        asthma = pd.read_csv('asthma.csv')

        asthma_sd = asthma[asthma['County'] == 'San Diego']
        asthma_sd["Zip_Code"] = asthma_sd["Zip_Code"].astype(str).str.zfill(5)
```

```
acs_zip = acs_zip[acs_zip["Zip_Code"].isin(asthma_sd["Zip_Code"].unique())]
acs_zip.head(10)
```

C:\Users\busti\AppData\Local\Temp\ipykernel\_27848\2224539920.py:5: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row\_indexer,col\_indexer] = value instead

See the caveats in the documentation: [https://pandas.pydata.org/pandas-docs/stable/user\\_guide/indexing.html#returning-a-view-versus-a-copy](https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy)

```
asthma_sd["Zip_Code"] = asthma_sd["Zip_Code"].astype(str).str.zfill(5)
```

Out[22]:

|  | pop_total | med_income | med_age | white_alone | black_alone | asian_alone | hispanic_ |
|--|-----------|------------|---------|-------------|-------------|-------------|-----------|
|--|-----------|------------|---------|-------------|-------------|-------------|-----------|

|              |         |          |      |         |       |       |     |
|--------------|---------|----------|------|---------|-------|-------|-----|
| <b>30905</b> | 18300.0 | 116600.0 | 42.5 | 14504.0 | 310.0 | 474.0 | 339 |
|--------------|---------|----------|------|---------|-------|-------|-----|

|              |         |          |      |        |       |        |     |
|--------------|---------|----------|------|--------|-------|--------|-----|
| <b>30906</b> | 17445.0 | 121727.0 | 46.9 | 7318.0 | 255.0 | 2578.0 | 824 |
|--------------|---------|----------|------|--------|-------|--------|-----|

|              |        |         |      |        |       |      |     |
|--------------|--------|---------|------|--------|-------|------|-----|
| <b>30908</b> | 3375.0 | 58561.0 | 44.9 | 2235.0 | 204.0 | 15.0 | 101 |
|--------------|--------|---------|------|--------|-------|------|-----|

|              |         |         |      |         |        |        |      |
|--------------|---------|---------|------|---------|--------|--------|------|
| <b>30909</b> | 77957.0 | 87705.0 | 38.4 | 28568.0 | 2888.0 | 8215.0 | 5012 |
|--------------|---------|---------|------|---------|--------|--------|------|

|              |         |         |      |         |        |        |      |
|--------------|---------|---------|------|---------|--------|--------|------|
| <b>30910</b> | 84619.0 | 79334.0 | 36.7 | 28566.0 | 3447.0 | 6106.0 | 6324 |
|--------------|---------|---------|------|---------|--------|--------|------|

|              |         |          |      |         |        |         |      |
|--------------|---------|----------|------|---------|--------|---------|------|
| <b>30911</b> | 57067.0 | 129412.0 | 35.6 | 14377.0 | 4767.0 | 15044.0 | 2700 |
|--------------|---------|----------|------|---------|--------|---------|------|

|              |         |          |      |        |       |        |     |
|--------------|---------|----------|------|--------|-------|--------|-----|
| <b>30912</b> | 17036.0 | 164398.0 | 38.4 | 6606.0 | 550.0 | 3065.0 | 891 |
|--------------|---------|----------|------|--------|-------|--------|-----|

|              |         |          |      |         |        |        |      |
|--------------|---------|----------|------|---------|--------|--------|------|
| <b>30913</b> | 35385.0 | 138283.0 | 34.9 | 10726.0 | 2853.0 | 8166.0 | 1540 |
|--------------|---------|----------|------|---------|--------|--------|------|

|              |         |         |      |         |        |        |      |
|--------------|---------|---------|------|---------|--------|--------|------|
| <b>30917</b> | 25897.0 | 79071.0 | 35.2 | 12669.0 | 1185.0 | 1765.0 | 1314 |
|--------------|---------|---------|------|---------|--------|--------|------|

|              |        |          |      |        |      |       |     |
|--------------|--------|----------|------|--------|------|-------|-----|
| <b>30919</b> | 9466.0 | 138953.0 | 45.7 | 7153.0 | 76.0 | 136.0 | 228 |
|--------------|--------|----------|------|--------|------|-------|-----|



In [23]: `acs_zip.to_csv("data/demographics_zipcode.csv", index=False)`

In [24]: `## Get Census Data by Tract`

```
ACS_VARS = {
    "B01003_001E": "pop_total",
    "B19013_001E": "med_income",
    "B01002_001E": "med_age",
    "B02001_002E": "white_alone",
    "B02001_003E": "black_alone",
    "B02001_005E": "asian_alone",
    "B03002_012E": "hispanic_any",
```

```
        "B17001_002E": "pop_below_poverty",
    }

    acs_cols = tuple(ACS_VARS.keys()) + ("NAME",)

    tracts_raw = c.acs5.state_county_tract(
        acs_cols,
        STATE_FIPS,
        SAN_DIEGO_COUNTY_FIPS,
        Census.ALL
    )

    acs = pd.DataFrame(tracts_raw)
    acs.rename(columns=ACS_VARS, inplace=True)

    acs["GEOID"] = acs["state"] + acs["county"] + acs["tract"]

    acs["p_white"] = acs["white_alone"] / acs["pop_total"]
    acs["p_black"] = acs["black_alone"] / acs["pop_total"]
    acs["p_asian"] = acs["asian_alone"] / acs["pop_total"]
    acs["p_hispanic"] = acs["hispanic_any"] / acs["pop_total"]
    acs["p_poverty"] = acs["pop_below_poverty"] / acs["pop_total"]

    acs.head()
```



Out[24]:

|          | pop_total | med_income | med_age | white_alone | black_alone | asian_alone | hispanic_any |
|----------|-----------|------------|---------|-------------|-------------|-------------|--------------|
| <b>0</b> | 2746.0    | 211023.0   | 52.5    | 2282.0      | 36.0        | 136.0       | 300.0        |
| <b>1</b> | 2376.0    | 110476.0   | 49.9    | 2082.0      | 0.0         | 136.0       | 82.0         |
| <b>2</b> | 4019.0    | 121264.0   | 41.9    | 3297.0      | 29.0        | 184.0       | 769.0        |
| <b>3</b> | 2487.0    | 79605.0    | 39.6    | 1771.0      | 22.0        | 131.0       | 531.0        |
| <b>4</b> | 2925.0    | 89850.0    | 44.0    | 2135.0      | 113.0       | 188.0       | 554.0        |

In [25]: *# Classify tracts as coastal or inland based on distance from tract centroids to th*

```

def get_tract_centroids(state_fips="06", county_fips="073"):
    layer = 10 # Census Tracts Layer
    url = (
        f"https://tigerweb.geo.census.gov/arcgis/rest/services/TIGERweb/Tracts_Bloc"
        f"?where=STATE+%3D+'{state_fips}'+AND+COUNTY+%3D+'{county_fips}'"
        f"&outFields=STATE,COUNTY,TRACT"
        f"&returnGeometry=true&geometryPrecision=6&outSR=4326"
        f"&f=json"
    )

    resp = requests.get(url).json()

    if "features" not in resp:
        print("API returned an error:", resp)
        raise ValueError("TIGERweb query failed")

```

```

rows = []

for feat in resp["features"]:
    attrs = feat["attributes"]
    geom = feat["geometry"]["rings"][0]

    xs = [p[0] for p in geom]
    ys = [p[1] for p in geom]
    lon = sum(xs) / len(xs)
    lat = sum(ys) / len(ys)

    rows.append({
        "STATE": attrs["STATE"],
        "COUNTY": attrs["COUNTY"],
        "TRACT": attrs["TRACT"],
        "centroid_lon": lon,
        "centroid_lat": lat
    })

return pd.DataFrame(rows)

centroids = get_tract_centroids()
centroids["GEOID"] = centroids["STATE"] + centroids["COUNTY"] + centroids["TRACT"]

```

```

In [26]: acs = acs.merge(centroids, on="GEOID", how="left")

acs["region"] = acs["centroid_lon"].apply(
    lambda x: "coastal" if x > -117.25 else "inland"
)

acs["region"].value_counts()

```

```

Out[26]: region
coastal    631
inland     106
Name: count, dtype: int64

```

```

In [27]: ## Checking for bad data

acs.isnull().sum().sum()

```

```

Out[27]: np.int64(10)

```

## EDA for Demographic Data

```

In [28]: acs.columns

```

```

Out[28]: Index(['pop_total', 'med_income', 'med_age', 'white_alone', 'black_alone',
               'asian_alone', 'hispanic_any', 'pop_below_poverty', 'NAME', 'state',
               'county', 'tract', 'GEOID', 'p_white', 'p_black', 'p_asian',
               'p_hispanic', 'p_poverty', 'STATE', 'COUNTY', 'TRACT', 'centroid_lon',
               'centroid_lat', 'region'],
              dtype='object')

```

In [29]: `acs.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 737 entries, 0 to 736
Data columns (total 24 columns):
#   Column                Non-Null Count  Dtype
---  -
0   pop_total              737 non-null   float64
1   med_income             737 non-null   float64
2   med_age                737 non-null   float64
3   white_alone            737 non-null   float64
4   black_alone            737 non-null   float64
5   asian_alone            737 non-null   float64
6   hispanic_any           737 non-null   float64
7   pop_below_poverty      737 non-null   float64
8   NAME                   737 non-null   object
9   state                  737 non-null   object
10  county                 737 non-null   object
11  tract                  737 non-null   object
12  GEOID                  737 non-null   object
13  p_white                735 non-null   float64
14  p_black                 735 non-null   float64
15  p_asian                 735 non-null   float64
16  p_hispanic              735 non-null   float64
17  p_poverty               735 non-null   float64
18  STATE                  737 non-null   object
19  COUNTY                 737 non-null   object
20  TRACT                  737 non-null   object
21  centroid_lon            737 non-null   float64
22  centroid_lat            737 non-null   float64
23  region                 737 non-null   object
dtypes: float64(15), object(9)
memory usage: 138.3+ KB
```

In [30]: `acs.describe()`

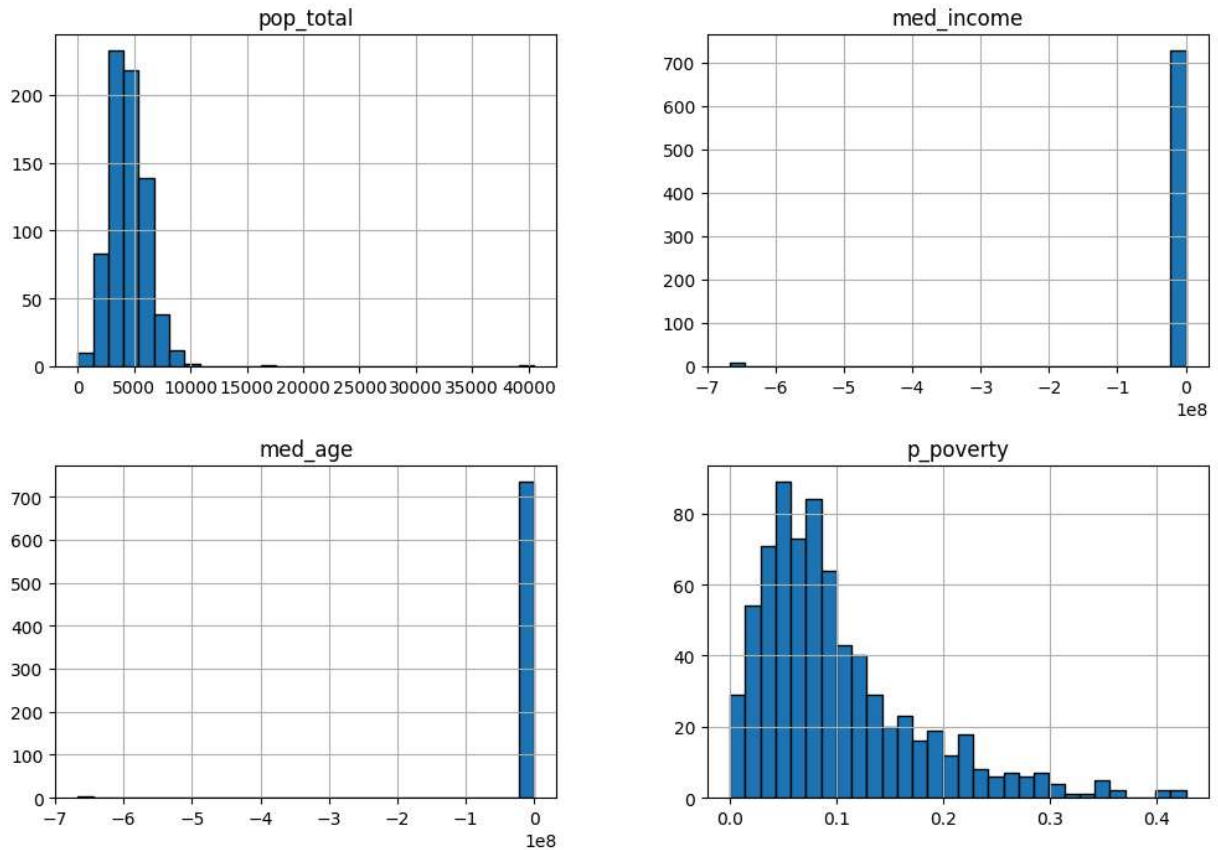
Out[30]:

|              | pop_total    | med_income    | med_age       | white_alone  | black_alone | asian_alone |
|--------------|--------------|---------------|---------------|--------------|-------------|-------------|
| <b>count</b> | 737.000000   | 7.370000e+02  | 7.370000e+02  | 737.000000   | 737.000000  | 737.000000  |
| <b>mean</b>  | 4454.249661  | -7.126849e+06 | -1.809097e+06 | 2361.221167  | 209.763908  | 544.175034  |
| <b>std</b>   | 2107.362836  | 6.913806e+07  | 3.470522e+07  | 1324.373218  | 290.434759  | 666.332208  |
| <b>min</b>   | 0.000000     | -6.666667e+08 | -6.666667e+08 | 0.000000     | 0.000000    | 0.000000    |
| <b>25%</b>   | 3313.000000  | 7.744400e+04  | 3.400000e+01  | 1586.000000  | 40.000000   | 134.000000  |
| <b>50%</b>   | 4257.000000  | 1.036170e+05  | 3.840000e+01  | 2202.000000  | 107.000000  | 300.000000  |
| <b>75%</b>   | 5436.000000  | 1.343580e+05  | 4.270000e+01  | 2893.000000  | 269.000000  | 645.000000  |
| <b>max</b>   | 40481.000000 | 2.500010e+05  | 7.310000e+01  | 22239.000000 | 3760.000000 | 5653.000000 |

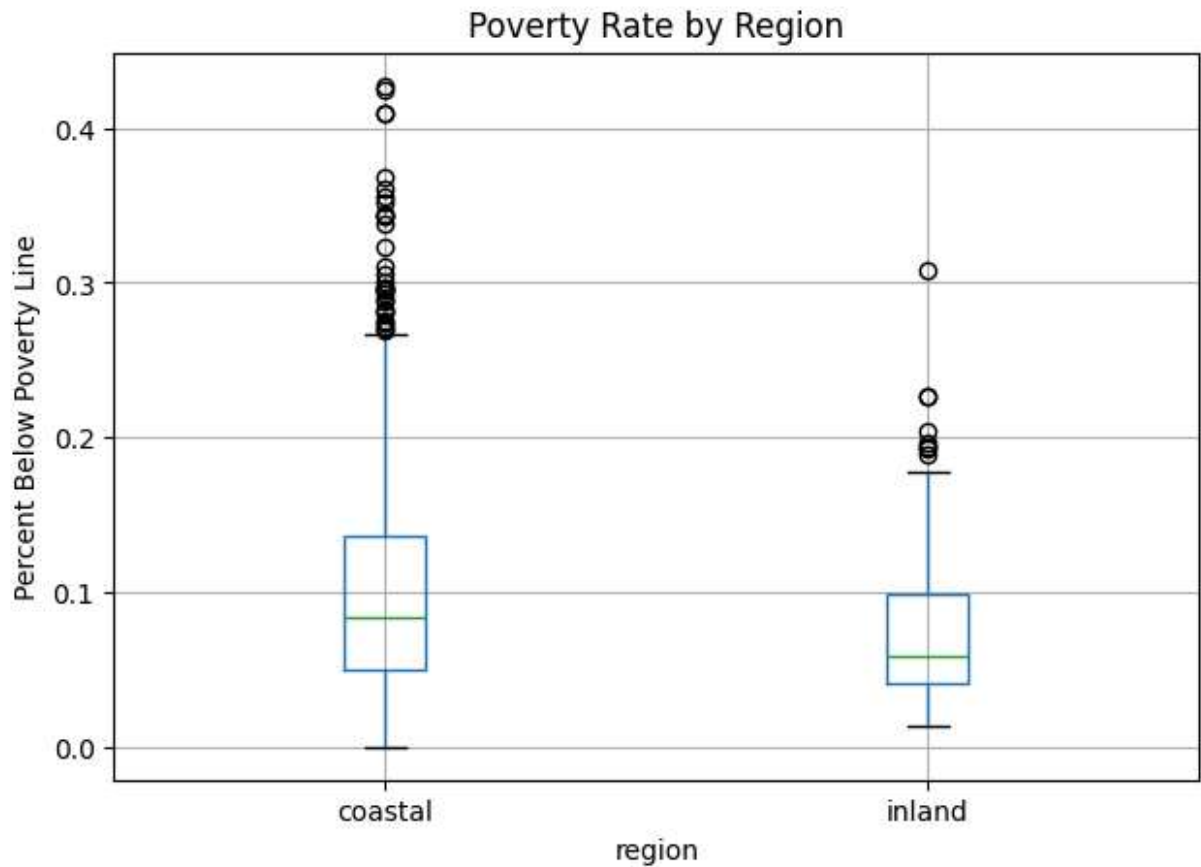
```
In [31]: num_cols = ["pop_total", "med_income", "med_age", "p_poverty"]

acs[num_cols].hist(bins=30, figsize=(12, 8), edgecolor="black")
plt.suptitle("Distribution of Key ACS Variables", fontsize=16)
plt.show()
```

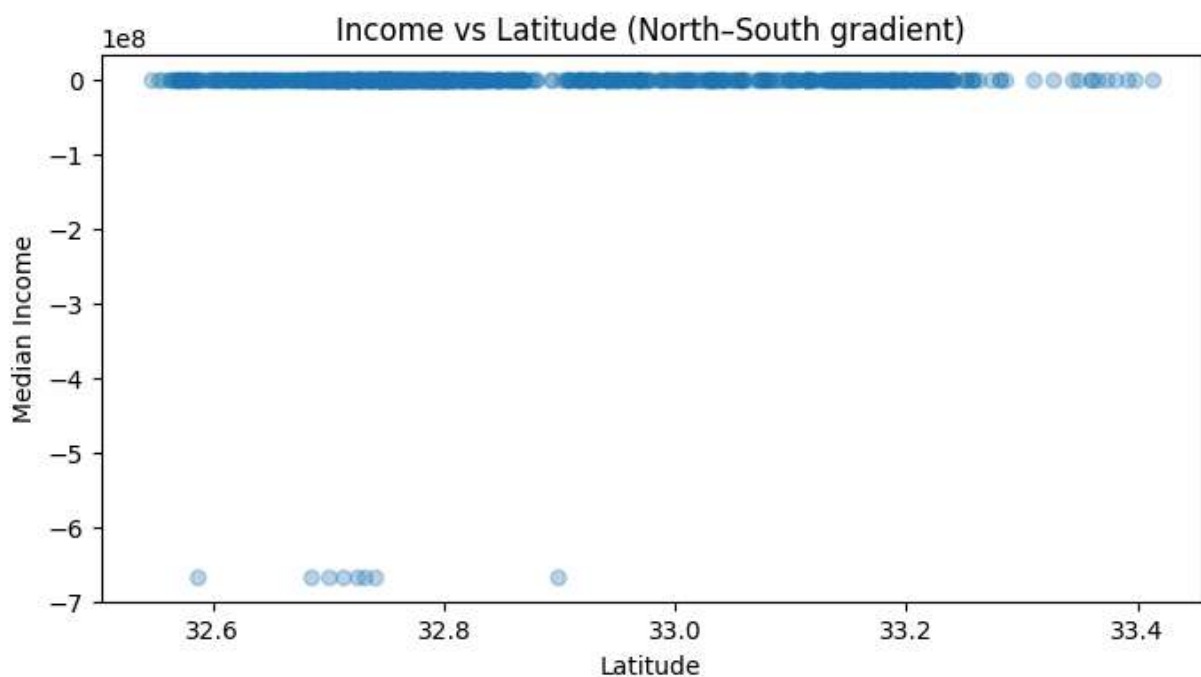
Distribution of Key ACS Variables



```
In [32]: acs.boxplot(column="p_poverty", by="region", figsize=(7,5))
plt.title("Poverty Rate by Region")
plt.suptitle("")
plt.ylabel("Percent Below Poverty Line")
plt.show()
```



```
In [33]: plt.scatter(acs["centroid_lat"], acs["med_income"], alpha=0.3)
plt.xlabel("Latitude")
plt.ylabel("Median Income")
plt.title("Income vs Latitude (North-South gradient)")
plt.show()
```



```
In [34]: acs[(acs["med_income"] < 0) | (acs["med_age"] < 0)]
```

Out[34]:

|            | pop_total | med_income   | med_age      | white_alone | black_alone | asian_alone | hispanic |
|------------|-----------|--------------|--------------|-------------|-------------|-------------|----------|
| <b>90</b>  | 8774.0    | -666666666.0 | 24.8         | 4492.0      | 1621.0      | 1060.0      | 20       |
| <b>115</b> | 295.0     | -666666666.0 | 20.9         | 119.0       | 130.0       | 0.0         |          |
| <b>124</b> | 31.0      | -666666666.0 | 20.5         | 16.0        | 7.0         | 0.0         |          |
| <b>125</b> | 2490.0    | -666666666.0 | 20.3         | 1542.0      | 221.0       | 141.0       | 8        |
| <b>268</b> | 1462.0    | -666666666.0 | 30.1         | 987.0       | 91.0        | 199.0       | 4        |
| <b>269</b> | 0.0       | -666666666.0 | -666666666.0 | 0.0         | 0.0         | 0.0         |          |
| <b>280</b> | 3217.0    | -666666666.0 | 41.6         | 1426.0      | 940.0       | 69.0        | 14       |
| <b>736</b> | 0.0       | -666666666.0 | -666666666.0 | 0.0         | 0.0         | 0.0         |          |

**pop\_total** **med\_income** **med\_age** **white\_alone** **black\_alone** **asian\_alone** **hispanic**

8 rows × 24 columns

While there are very few missing values, some tracts have too low of a population and/or its estimates for median income and median age were too unreliable, that's why a lot of these plots look so skewed, therefore the 8 rows above will be dropped.

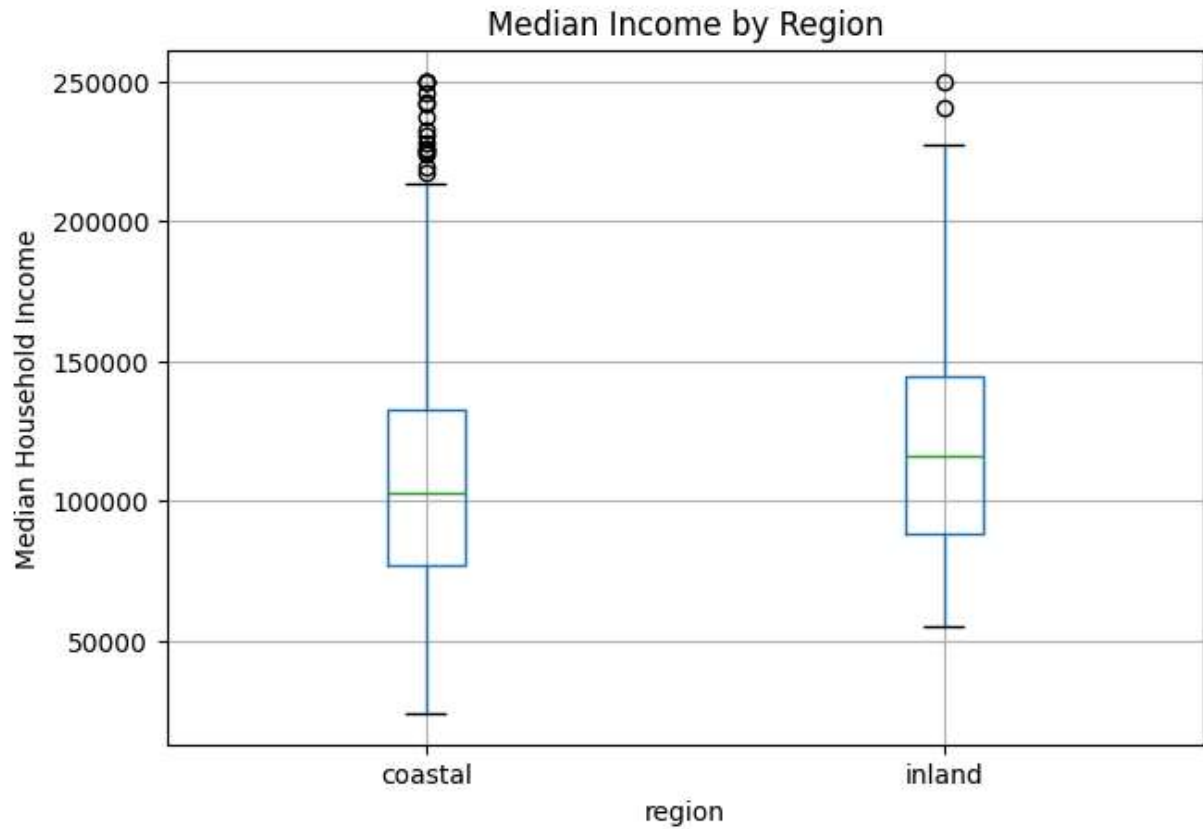
```
In [35]: bad_rows = acs[(acs['med_income'] < 0) | (acs['med_age'] < 0)]
         acs = acs[~((acs['med_income'] < 0) | (acs['med_age'] < 0))].copy()
```

```
In [36]: acs.describe()
```

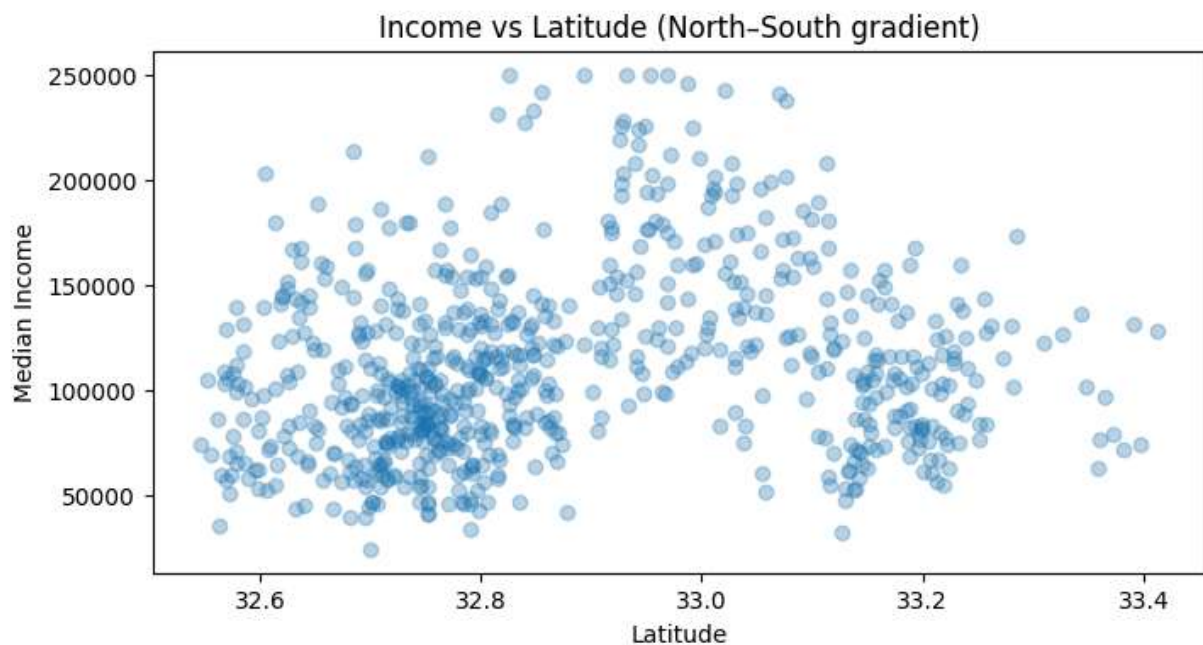
```
Out[36]:
```

|              | <b>pop_total</b> | <b>med_income</b> | <b>med_age</b> | <b>white_alone</b> | <b>black_alone</b> | <b>asian_alone</b> | <b>hispanic</b> |
|--------------|------------------|-------------------|----------------|--------------------|--------------------|--------------------|-----------------|
| <b>count</b> | 729.000000       | 729.000000        | 729.000000     | 729.000000         | 729.000000         | 729.000000         | 729.000000      |
| <b>mean</b>  | 4480.813443      | 110899.813443     | 38.995885      | 2375.360768        | 207.936900         | 548.131687         | 100.000000      |
| <b>std</b>   | 2082.929523      | 43742.453936      | 7.150089       | 1316.171214        | 285.660876         | 667.961366         | 0.000000        |
| <b>min</b>   | 889.000000       | 24125.000000      | 17.100000      | 240.000000         | 0.000000           | 0.000000           | 0.000000        |
| <b>25%</b>   | 3319.000000      | 78516.000000      | 34.100000      | 1596.000000        | 40.000000          | 135.000000         | 0.000000        |
| <b>50%</b>   | 4273.000000      | 104457.000000     | 38.500000      | 2208.000000        | 107.000000         | 305.000000         | 0.000000        |
| <b>75%</b>   | 5442.000000      | 135000.000000     | 42.700000      | 2909.000000        | 269.000000         | 647.000000         | 0.000000        |
| <b>max</b>   | 40481.000000     | 250001.000000     | 73.100000      | 22239.000000       | 3760.000000        | 5653.000000        | 1.000000        |

```
In [37]: acs.boxplot(column="med_income", by="region", figsize=(7,5))
         plt.title("Median Income by Region")
         plt.suptitle("")
         plt.ylabel("Median Household Income")
         plt.show()
```



```
In [38]: plt.scatter(acs["centroid_lat"], acs["med_income"], alpha=0.3)
plt.xlabel("Latitude")
plt.ylabel("Median Income")
plt.title("Income vs Latitude (North-South gradient)")
plt.show()
```



```
In [39]: plt.figure(figsize=(8, 5))
plt.scatter(
```



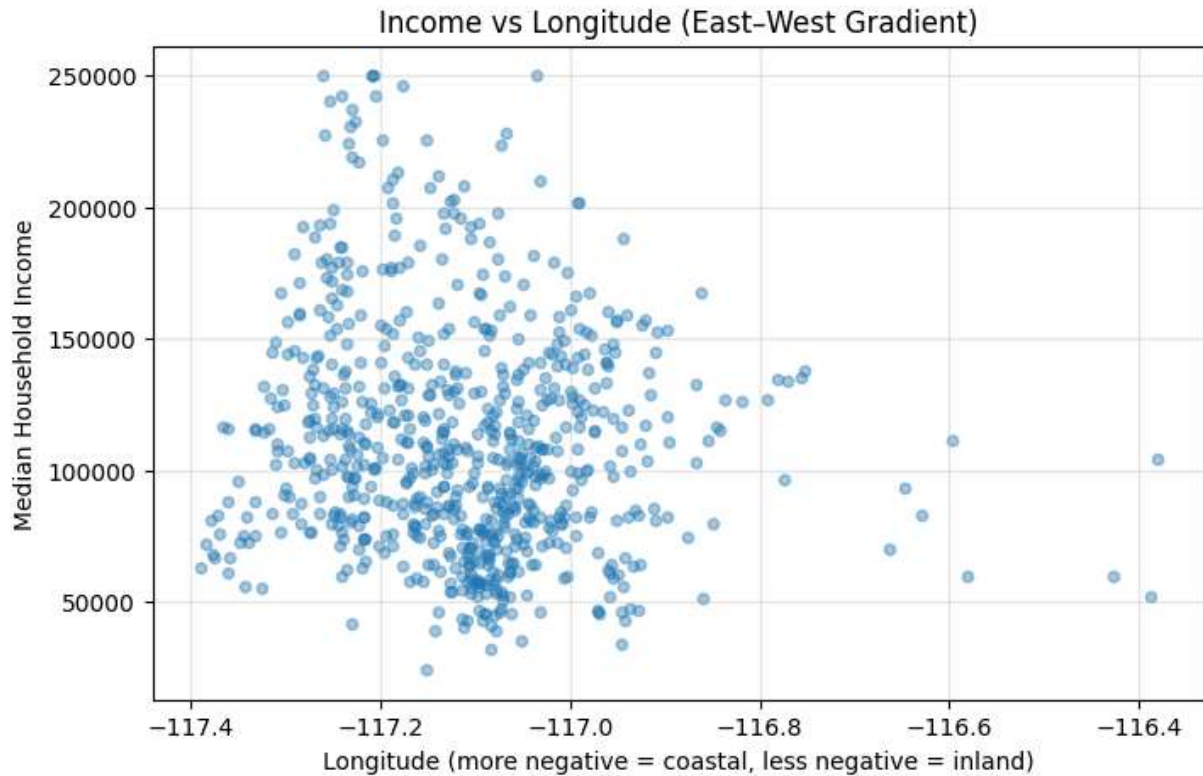
```

    acs["centroid_lon"],
    acs["med_income"],
    alpha=0.4,
    s=20
)

plt.xlabel("Longitude (more negative = coastal, less negative = inland)")
plt.ylabel("Median Household Income")
plt.title("Income vs Longitude (East-West Gradient)")

plt.grid(True, alpha=0.3)
plt.show()

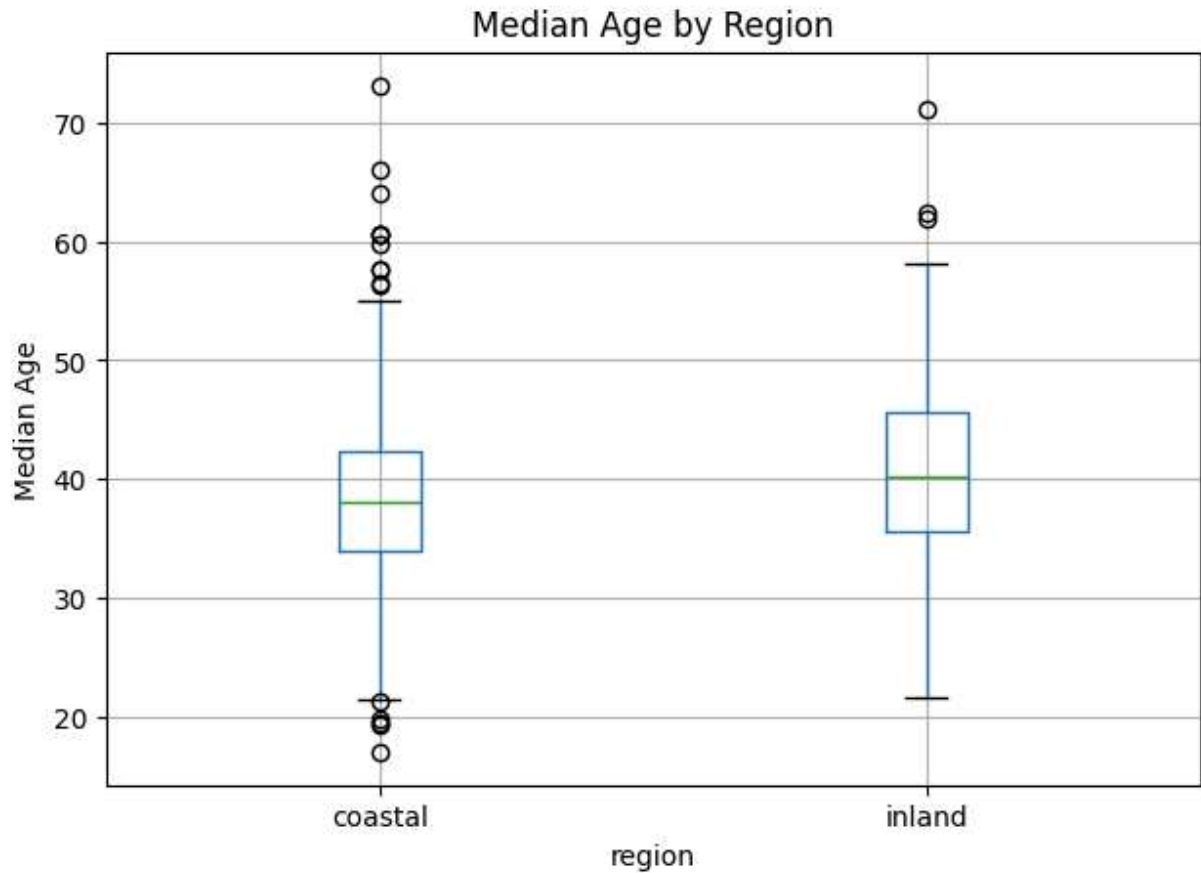
```



```

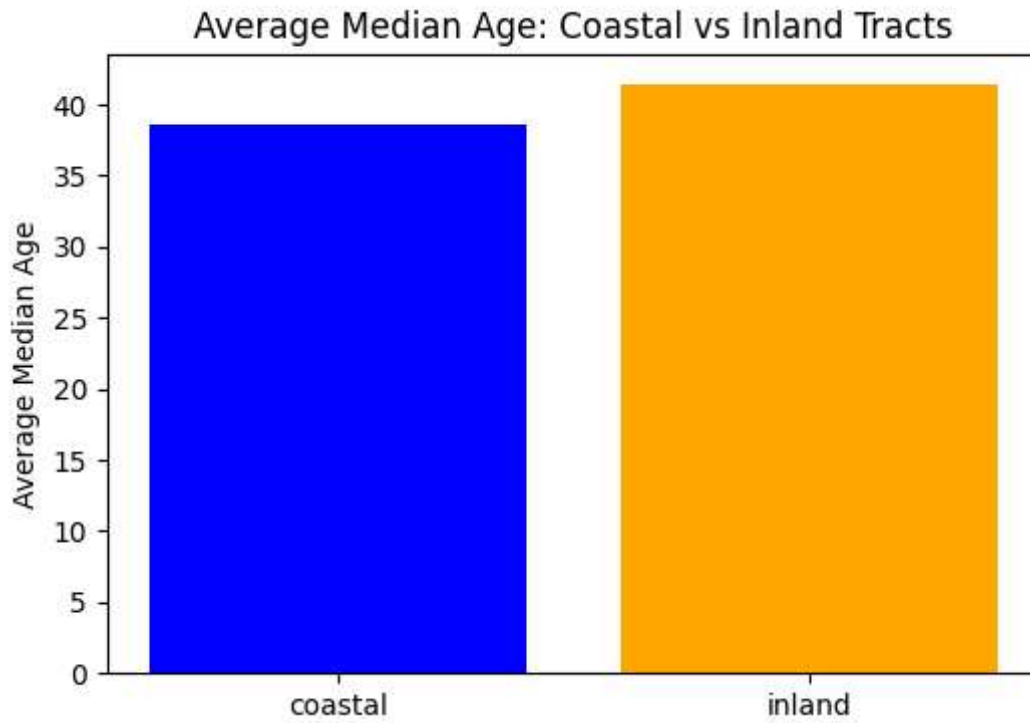
In [40]: acs.boxplot(column="med_age", by="region", figsize=(7,5))
plt.title("Median Age by Region")
plt.suptitle("")
plt.ylabel("Median Age")
plt.show()

```

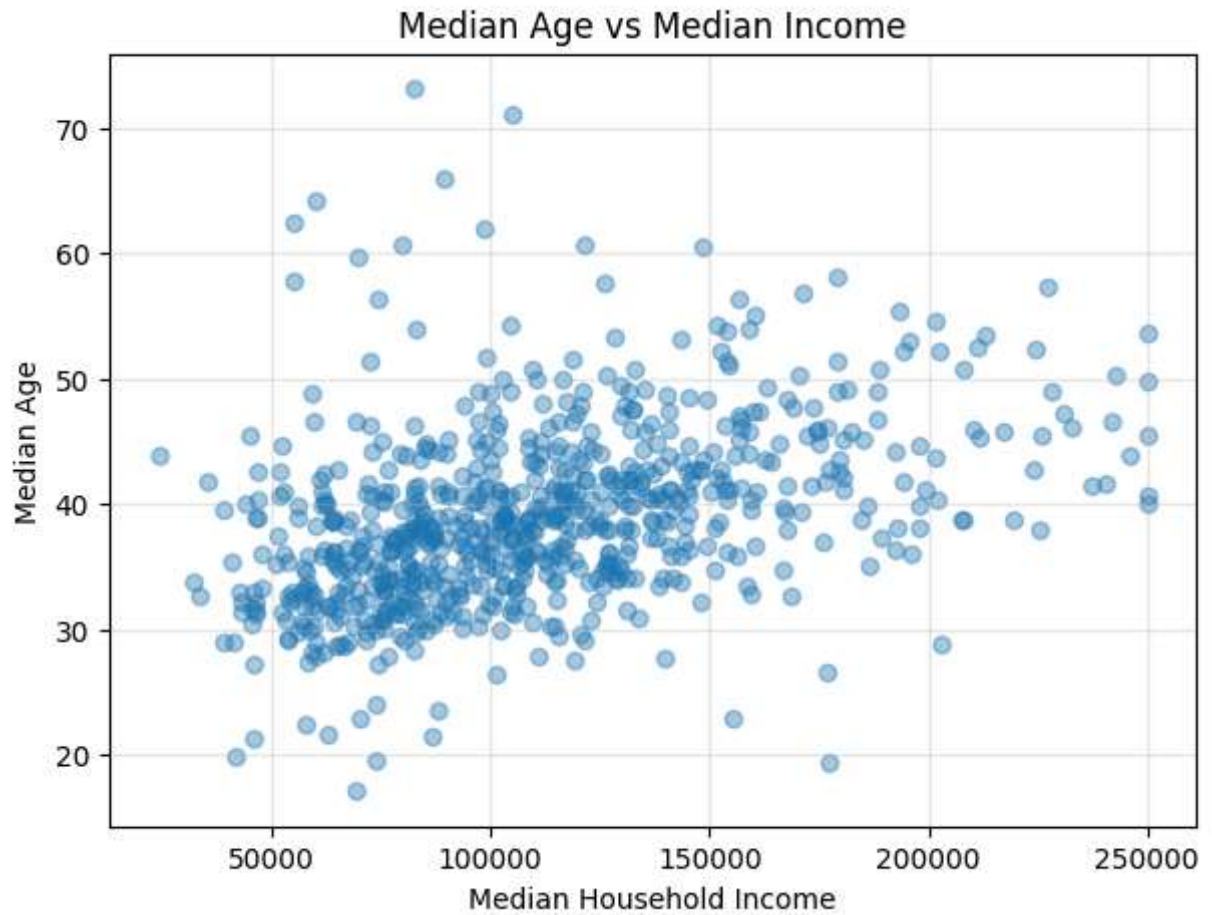


```
In [41]: avg_age = acs.groupby("region")["med_age"].mean().reset_index()

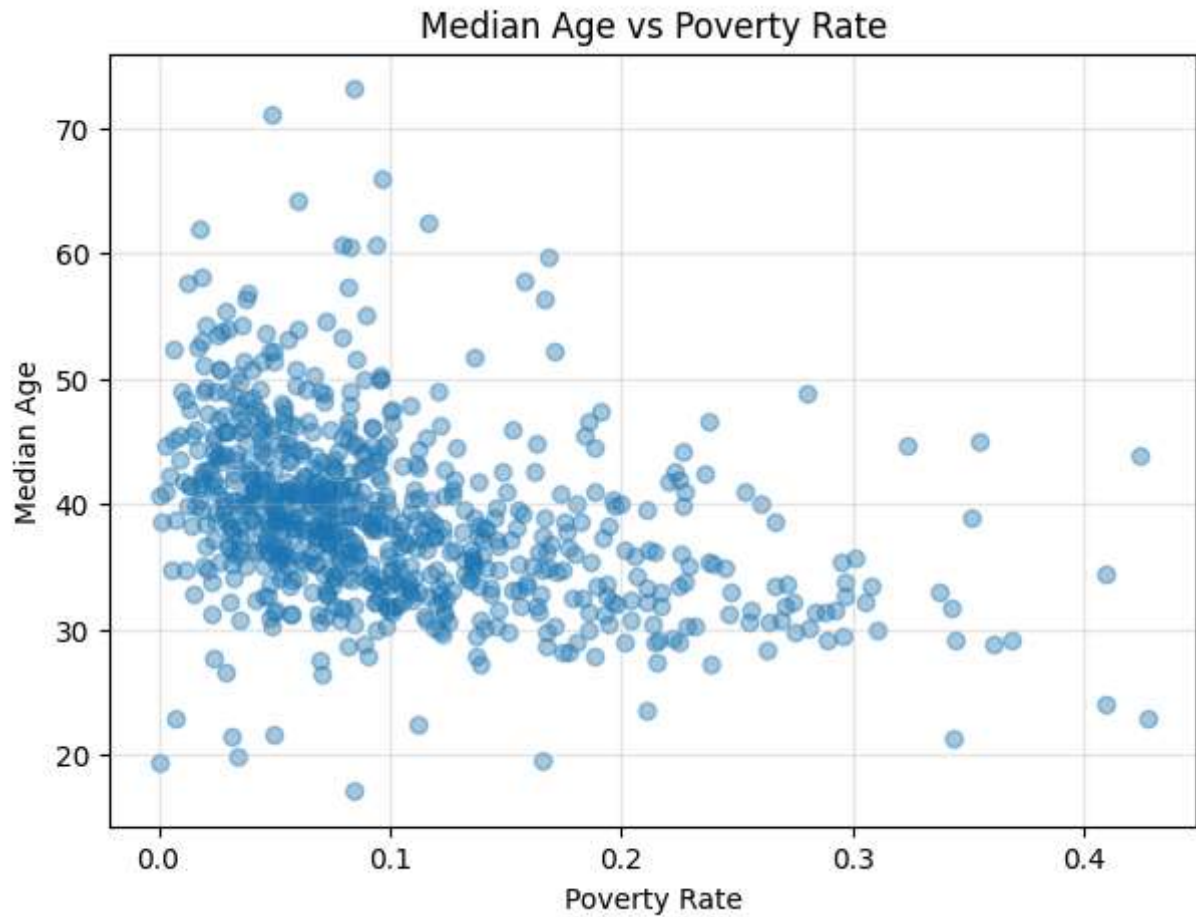
plt.figure(figsize=(6,4))
plt.bar(avg_age["region"], avg_age["med_age"], color=["blue", "orange"])
plt.ylabel("Average Median Age")
plt.title("Average Median Age: Coastal vs Inland Tracts")
plt.show()
```



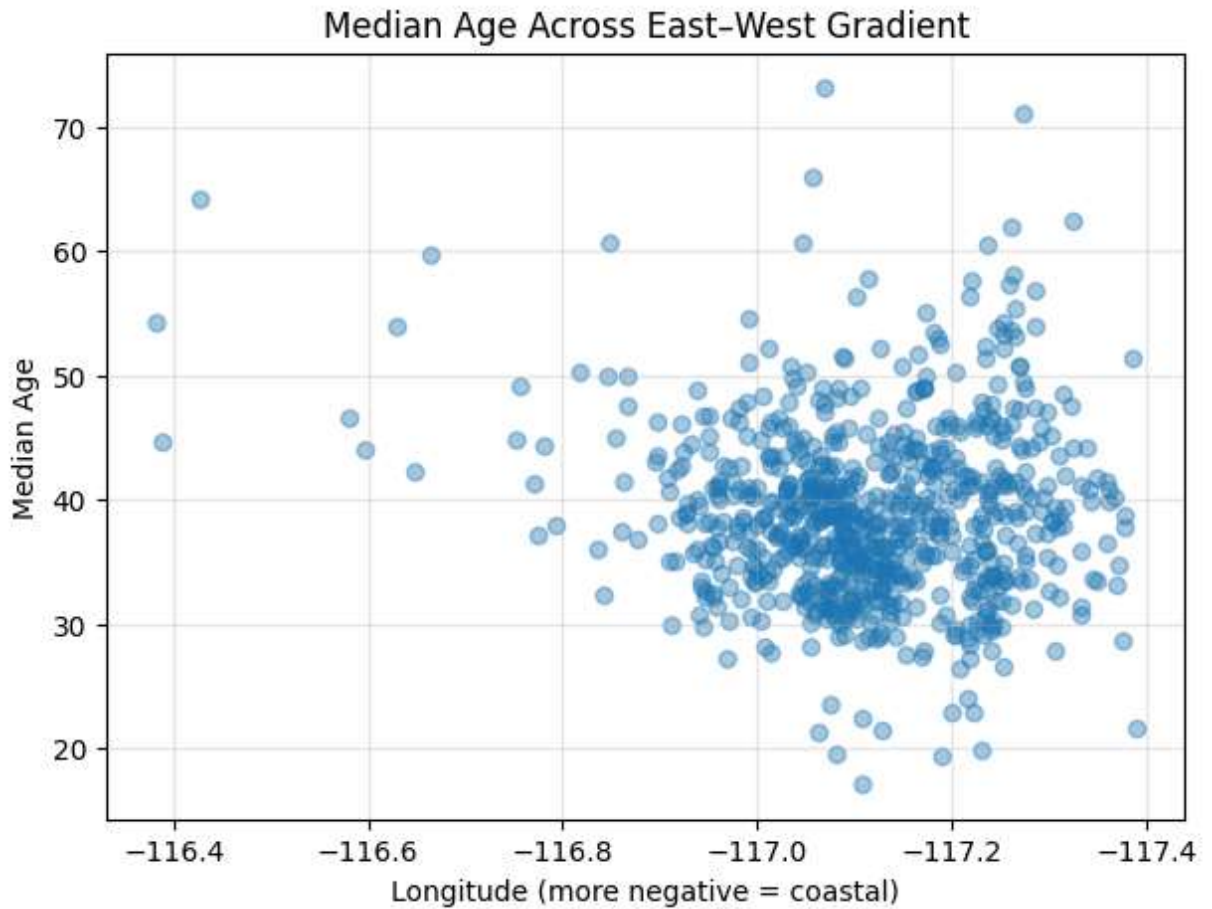
```
In [42]: plt.figure(figsize=(7,5))
plt.scatter(acs["med_income"], acs["med_age"], alpha=0.4)
plt.xlabel("Median Household Income")
plt.ylabel("Median Age")
plt.title("Median Age vs Median Income")
plt.grid(alpha=0.3)
plt.show()
```



```
In [43]: plt.figure(figsize=(7,5))
plt.scatter(acs["p_poverty"], acs["med_age"], alpha=0.4)
plt.xlabel("Poverty Rate")
plt.ylabel("Median Age")
plt.title("Median Age vs Poverty Rate")
plt.grid(alpha=0.3)
plt.show()
```



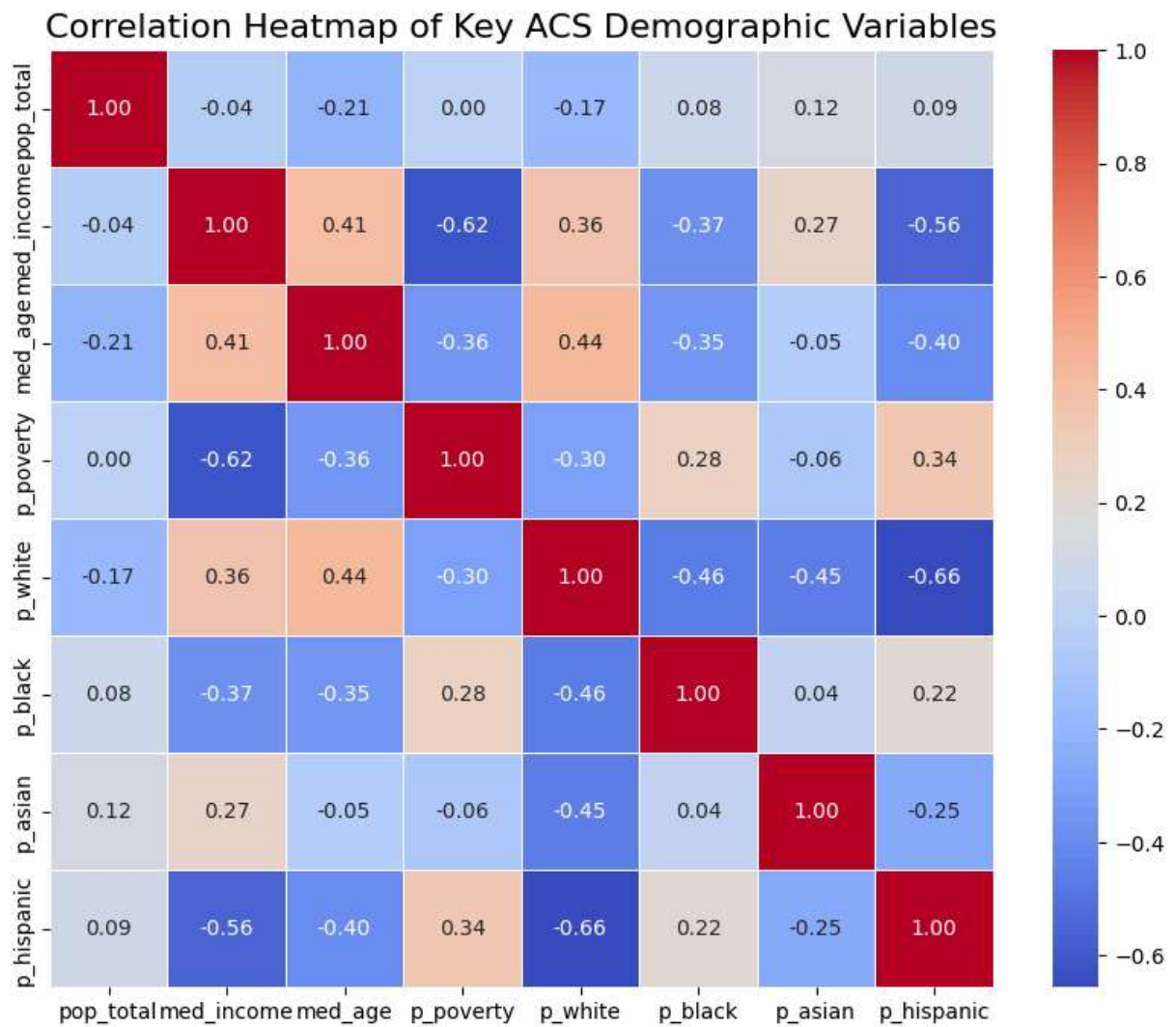
```
In [44]: plt.figure(figsize=(7,5))
plt.scatter(acs["centroid_lon"], acs["med_age"], alpha=0.4)
plt.xlabel("Longitude (more negative = coastal)")
plt.ylabel("Median Age")
plt.title("Median Age Across East-West Gradient")
plt.grid(alpha=0.3)
plt.gca().invert_xaxis() # makes coastline appear on the left
plt.show()
```



```
In [45]: corr_cols = [
    "pop_total",
    "med_income",
    "med_age",
    "p_poverty",
    "p_white",
    "p_black",
    "p_asian",
    "p_hispanic"
]

# Compute correlation matrix
corr_matrix = acs[corr_cols].corr()

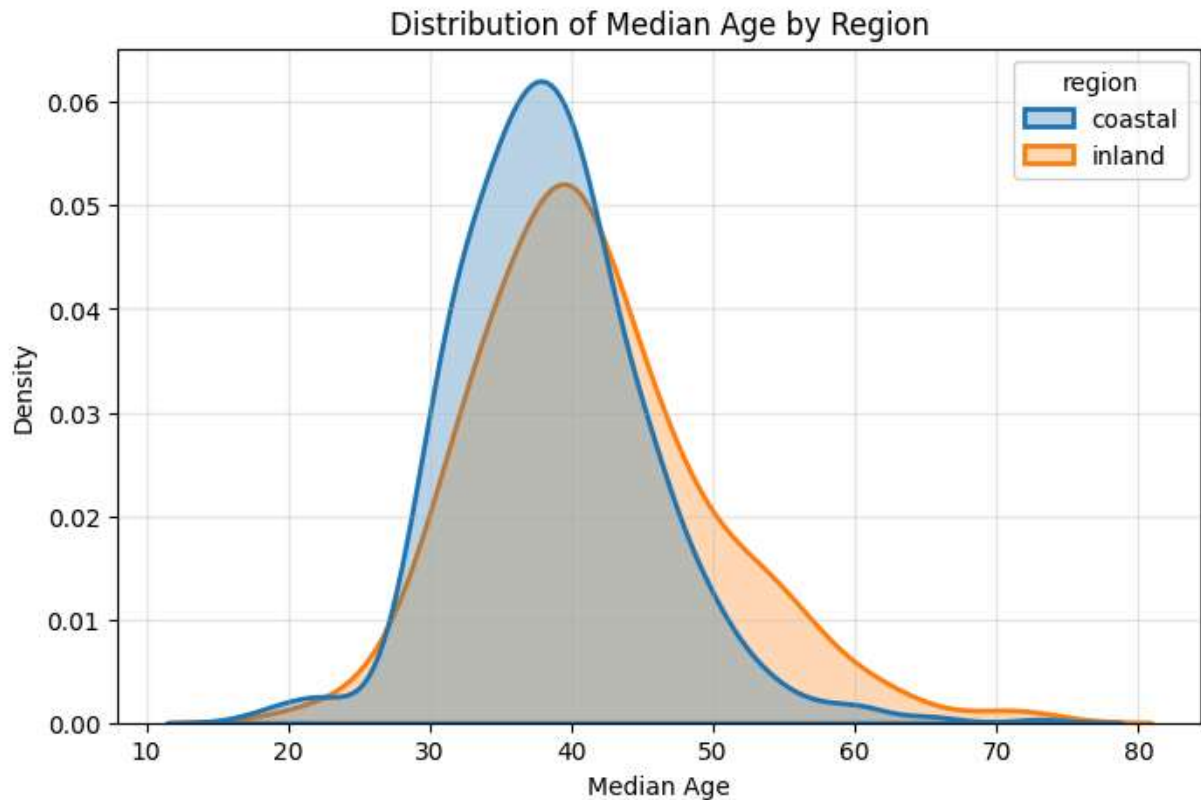
plt.figure(figsize=(10, 8))
sns.heatmap(
    corr_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)
plt.title("Correlation Heatmap of Key ACS Demographic Variables", fontsize=16)
plt.show()
```



```
In [46]: plt.figure(figsize=(8, 5))

sns.kdeplot(
    data=acs,
    x="med_age",
    hue="region",
    fill=True,
    common_norm=False,
    alpha=0.3,
    linewidth=2
)

plt.title("Distribution of Median Age by Region")
plt.xlabel("Median Age")
plt.ylabel("Density")
plt.grid(alpha=0.3)
plt.show()
```



```
In [47]: plt.figure(figsize=(7, 7))

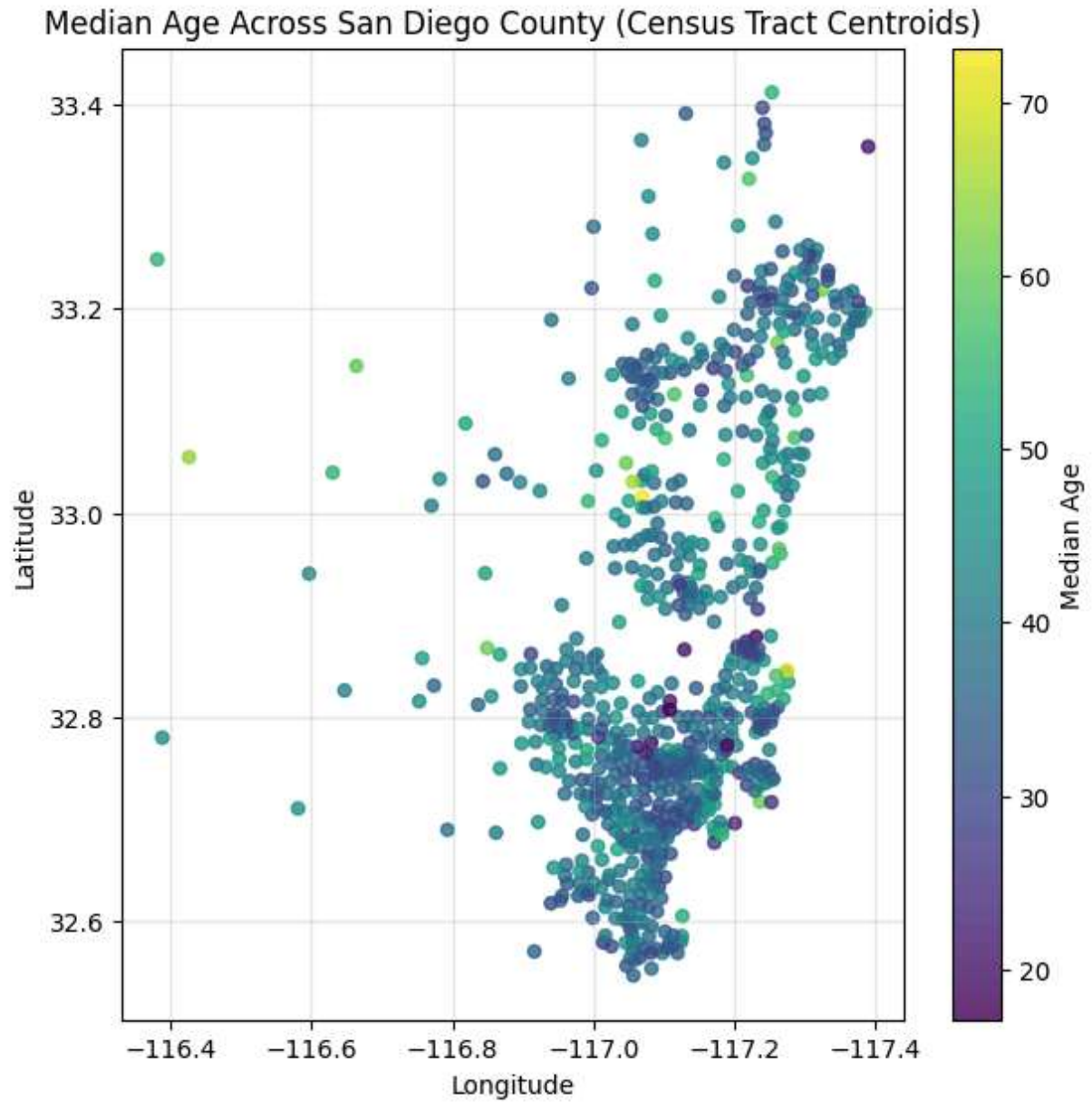
# Use color to represent age
scatter = plt.scatter(
    acs["centroid_lon"],
    acs["centroid_lat"],
    c=acs["med_age"],
    cmap="viridis",
    s=25,
    alpha=0.8
)

plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("Median Age Across San Diego County (Census Tract Centroids)")
plt.gca().invert_xaxis() # West (coast) on the left

# Colorbar Legend
cbar = plt.colorbar(scatter)
cbar.set_label("Median Age")

plt.grid(alpha=0.3)
plt.show()
```





```
In [48]: acs.isna().sum()
```

```
Out[48]: pop_total      0
         med_income    0
         med_age       0
         white_alone   0
         black_alone   0
         asian_alone   0
         hispanic_any  0
         pop_below_poverty 0
         NAME          0
         state         0
         county        0
         tract         0
         GEOID         0
         p_white       0
         p_black       0
         p_asian       0
         p_hispanic    0
         p_poverty     0
         STATE         0
         COUNTY        0
         TRACT         0
         centroid_lon  0
         centroid_lat  0
         region        0
         dtype: int64
```

```
In [49]: acs.to_csv("data/demographics.csv", index=False)
```

```
In [50]: acs['pop_total'].max()
```

```
Out[50]: np.float64(40481.0)
```

## Cleaning Asthma Data

```
In [51]: # Load in data
         asthma = pd.read_csv('asthma.csv')
```

```
In [52]: asthma.describe()
```

Out[52]:

|              | _id          | Year         | Zip_Code     | Number_of_Asthma_ED_Visits | Age_Adjusted_Rate_of_Asthma_ED_Visits |
|--------------|--------------|--------------|--------------|----------------------------|---------------------------------------|
| <b>count</b> | 21124.000000 | 21124.000000 | 21124.000000 |                            | 21124.000000                          |
| <b>mean</b>  | 10562.500000 | 2017.847235  | 93140.112431 |                            | 76.509468                             |
| <b>std</b>   | 6098.117879  | 3.171994     | 1777.452177  |                            | 73.104455                             |
| <b>min</b>   | 1.000000     | 2013.000000  | 90001.000000 |                            | 12.000000                             |
| <b>25%</b>   | 5281.750000  | 2015.000000  | 91784.000000 |                            | 27.000000                             |
| <b>50%</b>   | 10562.500000 | 2018.000000  | 93015.000000 |                            | 51.000000                             |
| <b>75%</b>   | 15843.250000 | 2021.000000  | 94611.000000 |                            | 101.000000                            |
| <b>max</b>   | 21124.000000 | 2023.000000  | 96161.000000 |                            | 800.000000                            |

In [53]: `asthma = asthma[(asthma['Year'] >= 2021) & (asthma['County'] == 'San Diego')]`

In [54]: `asthma["Zip_Code"].unique()`

Out[54]: `array([91901, 91910, 91911, 91913, 91915, 91932, 91941, 91942, 91945, 91950, 91977, 92019, 92020, 92021, 92024, 92025, 92026, 92027, 92028, 92040, 92054, 92056, 92057, 92058, 92064, 92065, 92069, 92071, 92078, 92081, 92083, 92084, 92101, 92102, 92104, 92105, 92111, 92113, 92114, 92115, 92117, 92119, 92120, 92123, 92124, 92126, 92127, 92128, 92129, 92139, 92154, 92173, 91902, 91906, 92008, 92011, 92103, 92107, 92108, 92109, 92110, 92116, 92118, 92122, 92130, 92131, 91914, 91935, 91978, 92009, 92029, 92037, 92082, 92106])`

In [55]: `asthma.isna().sum()`

Out[55]:

|                                  |   |
|----------------------------------|---|
| _id                              | 0 |
| Year                             | 0 |
| Zip_Code                         | 0 |
| Number_of_Asthma_ED_Visits       | 0 |
| Age_Adjusted_Rate_of_Asthma_ED_V | 0 |
| Age_Group                        | 0 |
| County                           | 0 |
| dtype: int64                     |   |

In [56]: `asthma['Zip_Code'].nunique()`

Out[56]: 74

## Merge Asthma and Demographics Data

In [57]: `demo = pd.read_csv('data/demographics_zipcode.csv')`

In [58]: `demo.columns`

```
Out[58]: Index(['pop_total', 'med_income', 'med_age', 'white_alone', 'black_alone',
               'asian_alone', 'hispanic_any', 'pop_below_poverty', 'NAME', 'Zip_Code',
               'p_white', 'p_black', 'p_asian', 'p_hispanic', 'p_poverty'],
              dtype='object')
```

```
In [59]: asthma = asthma.copy()
         demo = demo.copy()

         asthma["Zip_Code"] = asthma["Zip_Code"].astype(str).str.strip().str.zfill(5)
         demo["Zip_Code"] = demo["Zip_Code"].astype(str).str.strip().str.zfill(5)

         asthma_zips = set(asthma["Zip_Code"].unique())
         demo_zips = set(demo["Zip_Code"].unique())

         print("Asthma ZIPs:", len(asthma_zips))
         print("Census ZIPs:", len(demo_zips))
         print("Asthma ZIPs missing census data:", len(asthma_zips - demo_zips))
```

Asthma ZIPs: 74

Census ZIPs: 78

Asthma ZIPs missing census data: 0

```
In [60]: merged = asthma.merge(
         demo,
         on="Zip_Code",
         how="left",
         validate="many_to_one"
         )

         assert merged.shape[0] == asthma.shape[0]
         merged.head()
```

```
Out[60]:
```

|  | _id | Year | Zip_Code | Number_of_Asthma_ED_Visits | Age_Adjusted_Rate_of_Asthma_ED_Visits |
|--|-----|------|----------|----------------------------|---------------------------------------|
|--|-----|------|----------|----------------------------|---------------------------------------|

|   |       |      |       |    |      |
|---|-------|------|-------|----|------|
| 0 | 15851 | 2021 | 91901 | 23 | 73.7 |
|---|-------|------|-------|----|------|

|   |       |      |       |    |      |
|---|-------|------|-------|----|------|
| 1 | 15852 | 2021 | 91910 | 53 | 31.2 |
|---|-------|------|-------|----|------|

|   |       |      |       |    |      |
|---|-------|------|-------|----|------|
| 2 | 15853 | 2021 | 91911 | 70 | 33.0 |
|---|-------|------|-------|----|------|

|   |       |      |       |    |      |
|---|-------|------|-------|----|------|
| 3 | 15854 | 2021 | 91913 | 34 | 21.6 |
|---|-------|------|-------|----|------|

|   |       |      |       |    |      |
|---|-------|------|-------|----|------|
| 4 | 15855 | 2021 | 91915 | 21 | 21.5 |
|---|-------|------|-------|----|------|

5 rows × 21 columns

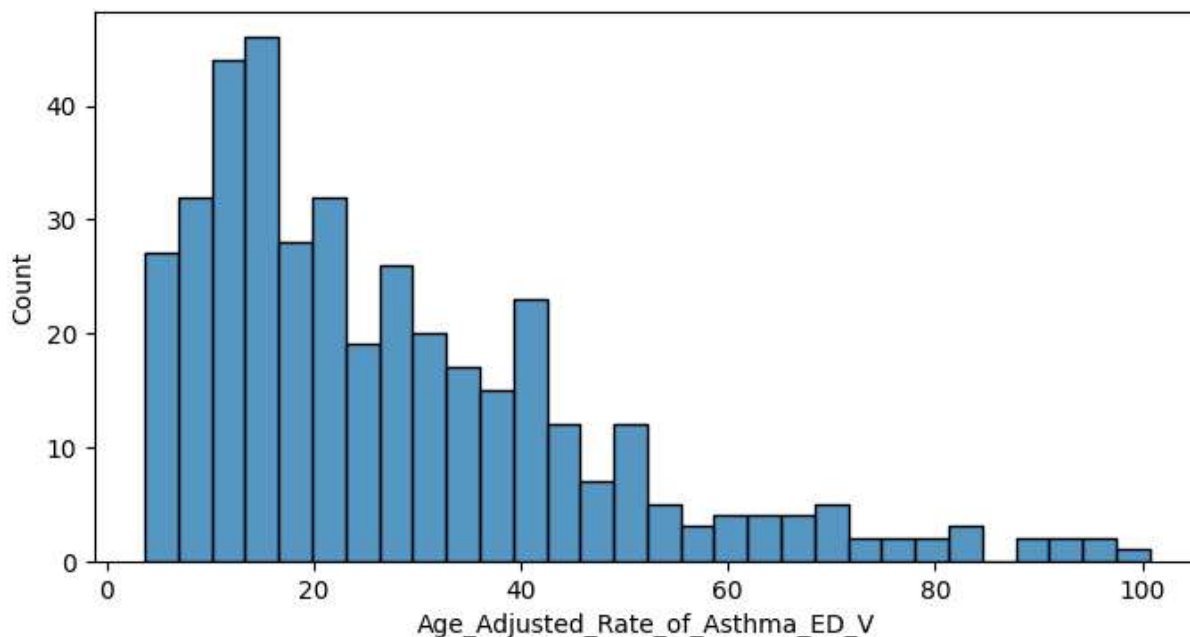


```
In [61]: merged.columns
```

```
Out[61]: Index(['_id', 'Year', 'Zip_Code', 'Number_of_Asthma_ED_Visits',
              'Age_Adjusted_Rate_of_Asthma_ED_V', 'Age_Group', 'County', 'pop_total',
              'med_income', 'med_age', 'white_alone', 'black_alone', 'asian_alone',
              'hispanic_any', 'pop_below_poverty', 'NAME', 'p_white', 'p_black',
              'p_asian', 'p_hispanic', 'p_poverty'],
              dtype='object')
```

```
In [62]: sns.histplot(merged["Age_Adjusted_Rate_of_Asthma_ED_V"], bins=30)
```

```
Out[62]: <Axes: xlabel='Age_Adjusted_Rate_of_Asthma_ED_V', ylabel='Count'>
```



```
In [63]: aqi_df['Local Site Name'].unique()
```

```
Out[63]: array(['Chula Vista', 'Alpine', 'Camp Pendleton', 'Donovan',
              'San Diego - Kearny Villa Rd.',
              'El Cajon - Lexington Elementary School',
              'San Diego - Sherman Elementary School', 'Pala Airpad',
              'La Posta Band of Indians'], dtype=object)
```

```
In [64]: ## Assign each zipcode to the closest AQI measuring station
demo["Zip_Code"] = demo["Zip_Code"].astype(str).str.strip().str.zfill(5)
asthma["Zip_Code"] = asthma["Zip_Code"].astype(str).str.strip().str.zfill(5)

# Only need centroids for ZIPs you actually have
zips = sorted(asthma["Zip_Code"].unique())

nomi = pgeocode.Nominatim("us")
geo = nomi.query_postal_code(zips)

zip_points = pd.DataFrame({
    "Zip_Code": zips,
    "centroid_lat": geo["latitude"].values,
    "centroid_lon": geo["longitude"].values
}).dropna(subset=["centroid_lat", "centroid_lon"])

print("ZIPs needed:", len(zips))
```

```
print("ZIPs with centroids found:", zip_points["Zip_Code"].nunique())
zip_points.head()
```

ZIPs needed: 74

ZIPs with centroids found: 74

Out[64]:

|   | Zip_Code | centroid_lat | centroid_lon |
|---|----------|--------------|--------------|
| 0 | 91901    | 32.8282      | -116.7543    |
| 1 | 91902    | 32.6671      | -117.0221    |
| 2 | 91906    | 32.6605      | -116.4905    |
| 3 | 91910    | 32.6371      | -117.0676    |
| 4 | 91911    | 32.6084      | -117.0565    |

In [65]:

```
aqi_stations = (
    aqi_df[["Site ID", "Local Site Name", "Site Latitude", "Site Longitude"]]
    .drop_duplicates()
    .rename(columns={
        "Site Latitude": "station_lat",
        "Site Longitude": "station_lon",
        "Local Site Name": "station_name",
        "Site ID": "station_id"
    })
    .reset_index(drop=True)
)

aqi_stations
```

Out[65]:

|   | station_id | station_name                           | station_lat | station_lon |
|---|------------|----------------------------------------|-------------|-------------|
| 0 | 60730001   | Chula Vista                            | 32.631242   | -117.059088 |
| 1 | 60731006   | Alpine                                 | 32.842318   | -116.768293 |
| 2 | 60731008   | Camp Pendleton                         | 33.217046   | -117.396249 |
| 3 | 60731014   | Donovan                                | 32.578165   | -116.921374 |
| 4 | 60731016   | San Diego - Kearny Villa Rd.           | 32.845709   | -117.123964 |
| 5 | 60731022   | El Cajon - Lexington Elementary School | 32.789565   | -116.944308 |
| 6 | 60731026   | San Diego - Sherman Elementary School  | 32.710177   | -117.142665 |
| 7 | 60731201   | Pala Airpad                            | 33.362593   | -117.090230 |
| 8 | 60739991   | La Posta Band of Indians               | 32.725189   | -116.364410 |

In [66]:

```
def haversine_km(lat1, lon1, lat2, lon2):
    R = 6371.0
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    dlat = lat2 - lat1
    dlon = lon2 - lon1
    a = np.sin(dlat/2)**2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon/2)**2
```

```

return 2 * R * np.arcsin(np.sqrt(a))

zip_points["_k"] = 1
aqi_stations["_k"] = 1
pairs = zip_points.merge(aqi_stations, on="_k").drop(columns="_k")

pairs["dist_km"] = haversine_km(
    pairs["centroid_lat"], pairs["centroid_lon"],
    pairs["station_lat"], pairs["station_lon"]
)

zip_to_station = (
    pairs.sort_values("dist_km")
        .groupby("Zip_Code", as_index=False)
        .first()[["Zip_Code", "station_id", "station_name", "dist_km"]]
)

zip_to_station.head()

```

Out[66]:

|   | Zip_Code | station_id | station_name             | dist_km   |
|---|----------|------------|--------------------------|-----------|
| 0 | 91901    | 60731006   | Alpine                   | 2.042945  |
| 1 | 91902    | 60730001   | Chula Vista              | 5.281132  |
| 2 | 91906    | 60739991   | La Posta Band of Indians | 13.819061 |
| 3 | 91910    | 60730001   | Chula Vista              | 1.029376  |
| 4 | 91911    | 60730001   | Chula Vista              | 2.551453  |

In [67]:

```

aqi_df["Date"] = pd.to_datetime(aqi_df["Date"])
aqi_df["Year"] = aqi_df["Date"].dt.year

station_year_aqi = (
    aqi_df.rename(columns={"Site ID": "station_id"})
        .groupby(["station_id", "Year"], as_index=False)["Daily AQI Value"]
        .mean()
        .rename(columns={"Daily AQI Value": "mean_aqi"})
)

station_year_aqi.head()

```

Out[67]:

|   | station_id | Year | mean_aqi  |
|---|------------|------|-----------|
| 0 | 60730001   | 2021 | 40.125698 |
| 1 | 60730001   | 2022 | 38.488506 |
| 2 | 60730001   | 2023 | 38.827298 |
| 3 | 60730001   | 2024 | 40.452722 |
| 4 | 60730001   | 2025 | 41.510067 |

```
In [68]: station_year_aqi["Year"].min(), station_year_aqi["Year"].max()
station_year_aqi["station_id"].nunique()
```

Out[68]: 9

```
In [69]: zip_year_aqi = zip_to_station.merge(station_year_aqi, on="station_id", how="left")
zip_year_aqi.head()
```

Out[69]:

|   | Zip_Code | station_id | station_name | dist_km  | Year | mean_aqi  |
|---|----------|------------|--------------|----------|------|-----------|
| 0 | 91901    | 60731006   | Alpine       | 2.042945 | 2021 | 58.421348 |
| 1 | 91901    | 60731006   | Alpine       | 2.042945 | 2022 | 59.253521 |
| 2 | 91901    | 60731006   | Alpine       | 2.042945 | 2023 | 55.456825 |
| 3 | 91901    | 60731006   | Alpine       | 2.042945 | 2024 | 58.446629 |
| 4 | 91901    | 60731006   | Alpine       | 2.042945 | 2025 | 52.620896 |

```
In [70]: zip_year_aqi.isna().sum().head()
```

Out[70]: Zip\_Code 0  
station\_id 0  
station\_name 0  
dist\_km 0  
Year 0  
dtype: int64

```
In [71]: final = merged.merge(
    zip_year_aqi[["Zip_Code", "Year", "mean_aqi", "station_name", "dist_km"]],
    on=["Zip_Code", "Year"],
    how="left",
    validate="many_to_one"
)

assert final.shape[0] == merged.shape[0]
final[["Zip_Code", "Year", "mean_aqi", "station_name", "dist_km"]].head()
```

Out[71]:

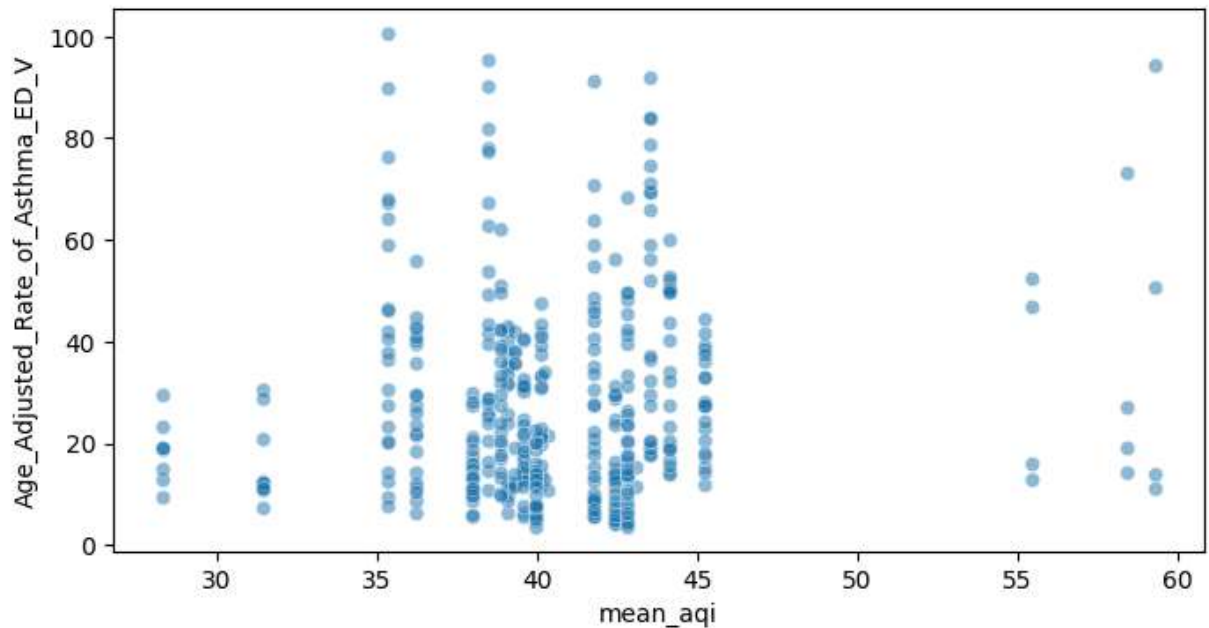
|   | Zip_Code | Year | mean_aqi  | station_name | dist_km  |
|---|----------|------|-----------|--------------|----------|
| 0 | 91901    | 2021 | 58.421348 | Alpine       | 2.042945 |
| 1 | 91910    | 2021 | 40.125698 | Chula Vista  | 1.029376 |
| 2 | 91911    | 2021 | 40.125698 | Chula Vista  | 2.551453 |
| 3 | 91913    | 2021 | 40.125698 | Chula Vista  | 7.268999 |
| 4 | 91915    | 2021 | 40.310924 | Donovan      | 6.203463 |

```
In [72]: # Drop missing values because there are very few of them and correspond to very rur
model_df = final.dropna(subset=["mean_aqi"]).copy()
```



```
In [73]: sns.scatterplot(
    data=model_df,
    x="mean_aqi",
    y="Age_Adjusted_Rate_of_Asthma_ED_V",
    alpha=0.5
)
```

Out[73]: <Axes: xlabel='mean\_aqi', ylabel='Age\_Adjusted\_Rate\_of\_Asthma\_ED\_V'>



```
In [74]: formula = (
    "Age_Adjusted_Rate_of_Asthma_ED_V ~ "
    "mean_aqi + med_income + p_poverty + p_hispanic + p_black + Year"
)

model = smf.ols(
    formula,
    data=model_df
).fit()

model.summary()
```

Out[74]:

## OLS Regression Results

|                   |                                  |                     |                       |                    |
|-------------------|----------------------------------|---------------------|-----------------------|--------------------|
| Dep. Variable:    | Age_Adjusted_Rate_of_Asthma_ED_V | R-squared:          | 0.195                 |                    |
| Model:            | OLS                              | Adj. R-squared:     | 0.183                 |                    |
| Method:           | Least Squares                    | F-statistic:        | 15.90                 |                    |
| Date:             | Fri, 12 Dec 2025                 | Prob (F-statistic): | 2.24e-16              |                    |
| Time:             | 23:24:27                         | Log-Likelihood:     | -1709.7               |                    |
| No. Observations: | 400                              | AIC:                | 3433.                 |                    |
| Df Residuals:     | 393                              | BIC:                | 3461.                 |                    |
| Df Model:         | 6                                |                     |                       |                    |
| Covariance Type:  | nonrobust                        |                     |                       |                    |
|                   | coef                             | std err             | t P> t  [0.025 0.975] |                    |
| Intercept         | -2200.5373                       | 2222.243            | -0.990 0.323          | -6569.508 2168.434 |
| mean_aqi          | 0.7842                           | 0.207               | 3.793 0.000           | 0.378 1.191        |
| med_income        | -5.65e-05                        | 5.51e-05            | -1.025 0.306          | -0.000 5.19e-05    |
| p_poverty         | 68.3897                          | 31.820              | 2.149 0.032           | 5.832 130.948      |
| p_hispanic        | 5.4010                           | 6.307               | 0.856 0.392           | -7.000 17.802      |
| p_black           | 138.6586                         | 26.522              | 5.228 0.000           | 86.517 190.800     |
| Year              | 1.0817                           | 1.099               | 0.984 0.326           | -1.079 3.242       |
| Omnibus:          | 90.949                           | Durbin-Watson:      | 0.605                 |                    |
| Prob(Omnibus):    | 0.000                            | Jarque-Bera (JB):   | 160.066               |                    |
| Skew:             | 1.305                            | Prob(JB):           | 1.75e-35              |                    |
| Kurtosis:         | 4.671                            | Cond. No.           | 2.78e+08              |                    |

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.  
 [2] The condition number is large, 2.78e+08. This might indicate that there are strong multicollinearity or other numerical problems.

Notes: [1] Standard Errors assume that the covariance matrix of the errors is correctly specified. [2] The condition number is large, 2.78e+08. This might indicate that there are strong multicollinearity or other numerical problems.

```
In [75]: ## Cook's Distance to test outlier influence
influence = model.get_influence()
```

```

cooks = influence.cooks_distance[0]

model_df["cooks_d"] = cooks
#model_df.sort_values("cooks_d", ascending=False).head()

cooks = model_df[model_df['mean_aqi'] > 50][['mean_aqi', 'cooks_d']]
cooks['Is_Influential'] = cooks['cooks_d'].apply(lambda x: x > (4/model_df.shape[0]
cooks

```

Out[75]:

|     | mean_aqi  | cooks_d  | Is_Influential |
|-----|-----------|----------|----------------|
| 0   | 58.421348 | 0.037658 | True           |
| 25  | 58.421348 | 0.000496 | False          |
| 52  | 58.421348 | 0.006721 | False          |
| 81  | 58.421348 | 0.007826 | False          |
| 118 | 59.253521 | 0.087215 | True           |
| 152 | 59.253521 | 0.008181 | False          |
| 190 | 59.253521 | 0.017367 | True           |
| 223 | 59.253521 | 0.009842 | False          |
| 260 | 55.456825 | 0.005852 | False          |
| 292 | 55.456825 | 0.004835 | False          |
| 329 | 55.456825 | 0.006907 | False          |
| 362 | 55.456825 | 0.006797 | False          |

## Regression Analysis Plots

In [76]: `model_df["fitted"] = model.fittedvalues`

In [77]:

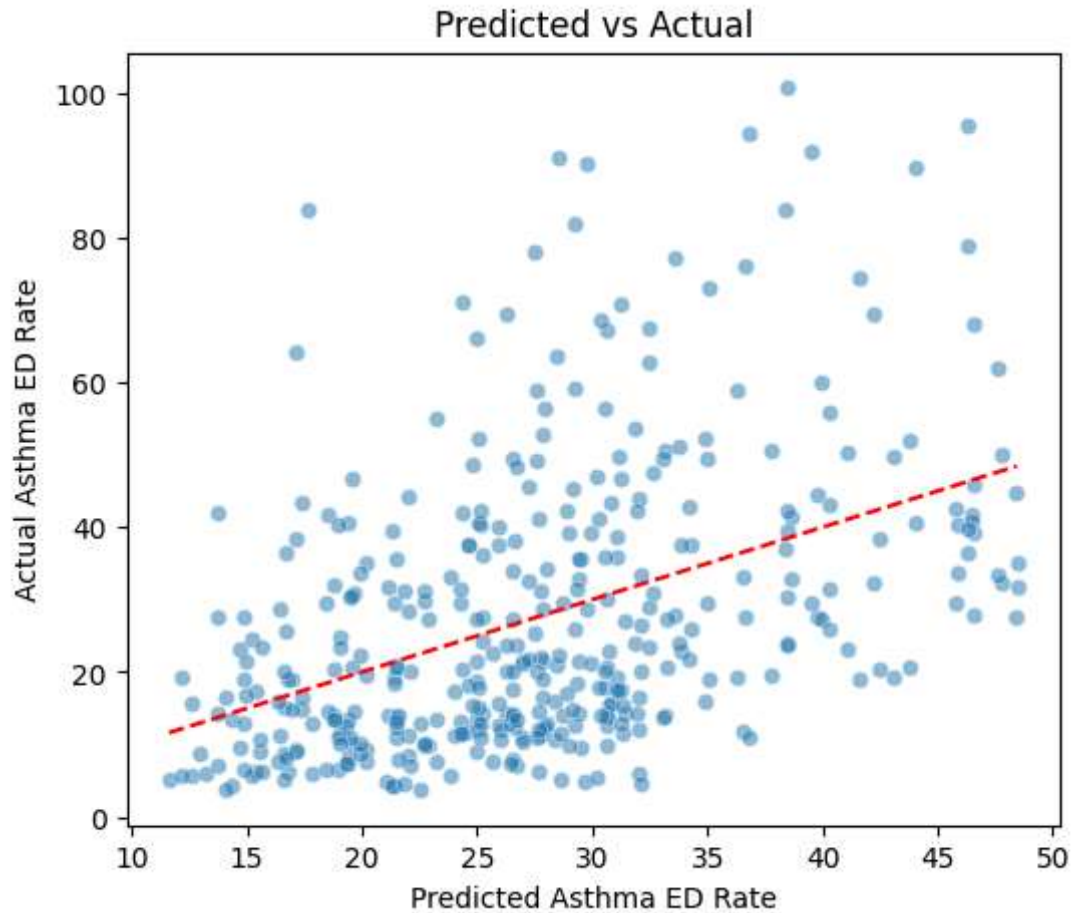
```

model_df["fitted"] = model.fittedvalues

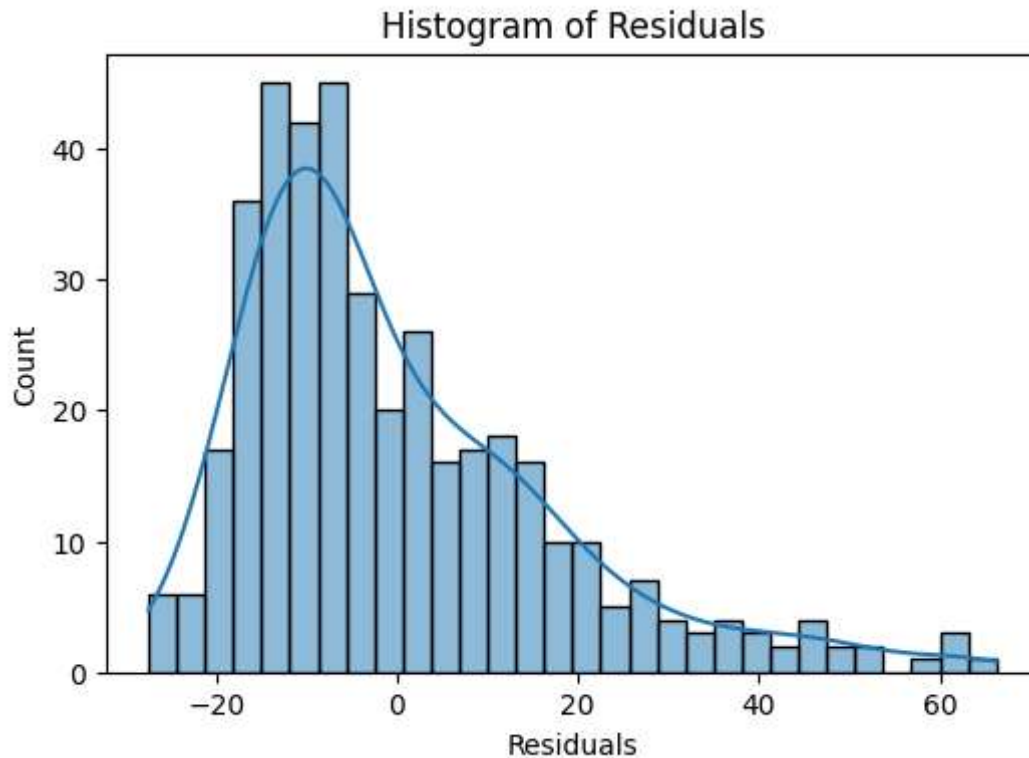
plt.figure(figsize=(6, 5))
sns.scatterplot(
    x=model_df["fitted"],
    y=model_df["Age_Adjusted_Rate_of_Asthma_ED_V"],
    alpha=0.5
)
plt.plot(
    [model_df["fitted"].min(), model_df["fitted"].max()],
    [model_df["fitted"].min(), model_df["fitted"].max()],
    color="red",
    linestyle="--"
)
plt.xlabel("Predicted Asthma ED Rate")
plt.ylabel("Actual Asthma ED Rate")

```

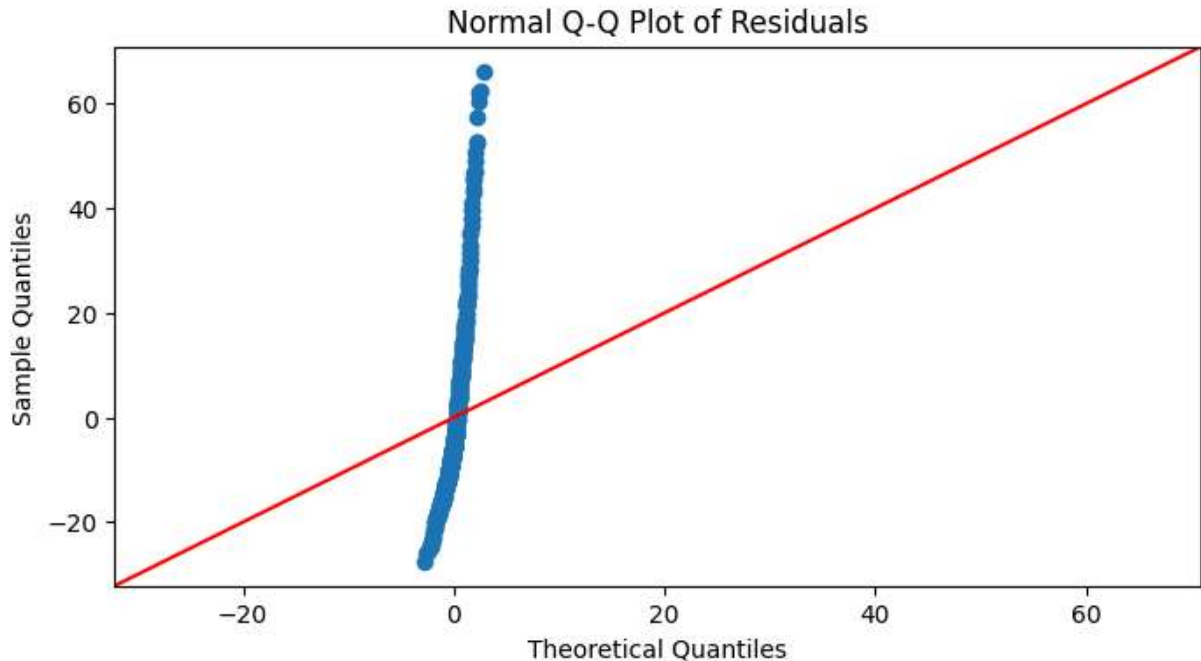
```
plt.title("Predicted vs Actual")  
plt.show()
```



```
In [78]: residuals = model.resid  
  
plt.figure(figsize=(6, 4))  
sns.histplot(residuals, bins=30, kde=True)  
plt.xlabel("Residuals")  
plt.title("Histogram of Residuals")  
plt.show()
```

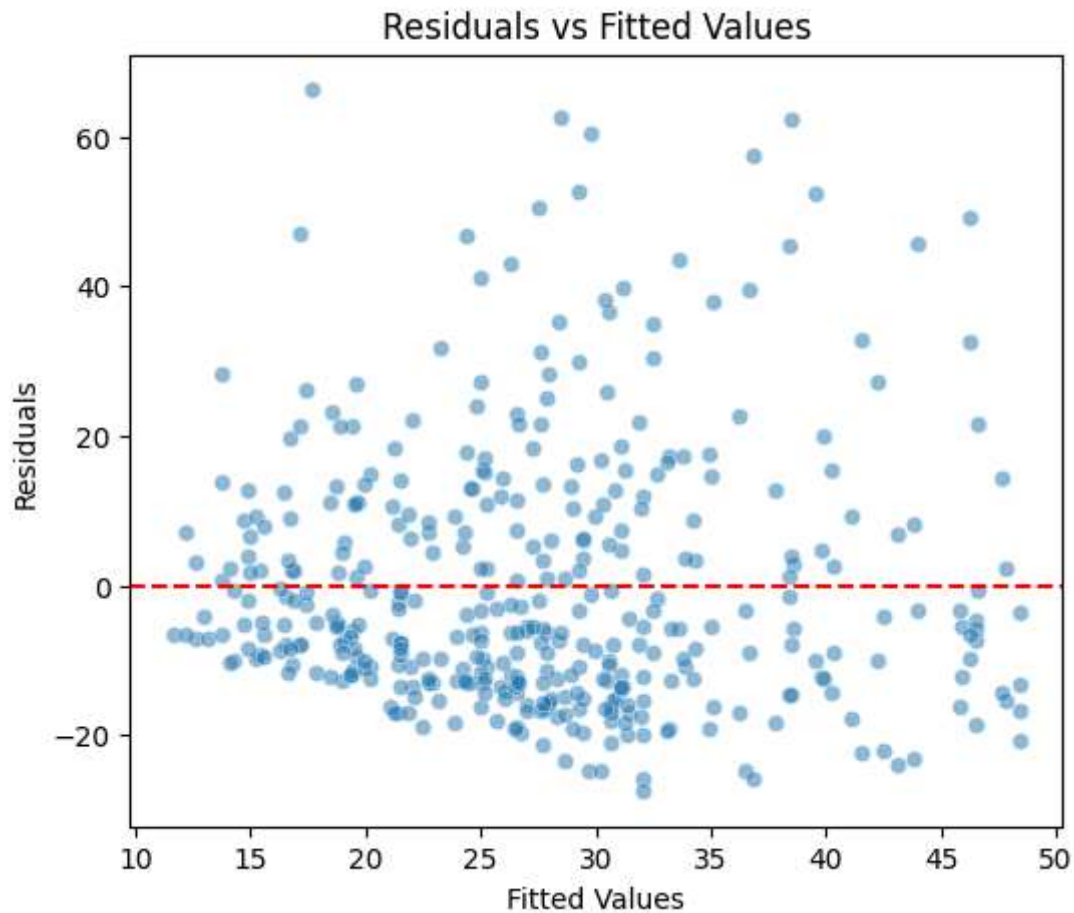


```
In [79]: sm.qqplot(residuals, line="45")
plt.title("Normal Q-Q Plot of Residuals")
plt.show()
```



```
In [80]: plt.figure(figsize=(6, 5))
sns.scatterplot(
    x=model_df["fitted"],
    y=residuals,
    alpha=0.5
)
plt.axhline(0, color="red", linestyle="--")
```

```
plt.xlabel("Fitted Values")
plt.ylabel("Residuals")
plt.title("Residuals vs Fitted Values")
plt.show()
```



```
In [81]: model_robust = smf.ols(formula, data=model_df).fit(cov_type="HC3")
model_robust.summary()
```

Out[81]:

## OLS Regression Results

|                          |                                  |                            |          |
|--------------------------|----------------------------------|----------------------------|----------|
| <b>Dep. Variable:</b>    | Age_Adjusted_Rate_of_Asthma_ED_V | <b>R-squared:</b>          | 0.195    |
| <b>Model:</b>            | OLS                              | <b>Adj. R-squared:</b>     | 0.183    |
| <b>Method:</b>           | Least Squares                    | <b>F-statistic:</b>        | 17.46    |
| <b>Date:</b>             | Fri, 12 Dec 2025                 | <b>Prob (F-statistic):</b> | 6.16e-18 |
| <b>Time:</b>             | 23:24:29                         | <b>Log-Likelihood:</b>     | -1709.7  |
| <b>No. Observations:</b> | 400                              | <b>AIC:</b>                | 3433.    |
| <b>Df Residuals:</b>     | 393                              | <b>BIC:</b>                | 3461.    |
| <b>Df Model:</b>         | 6                                |                            |          |
| <b>Covariance Type:</b>  | HC3                              |                            |          |

|                   | coef       | std err  | z      | P> z  | [0.025    | 0.975]   |
|-------------------|------------|----------|--------|-------|-----------|----------|
| <b>Intercept</b>  | -2200.5373 | 1654.958 | -1.330 | 0.184 | -5444.196 | 1043.121 |
| <b>mean_aqi</b>   | 0.7842     | 0.262    | 2.998  | 0.003 | 0.271     | 1.297    |
| <b>med_income</b> | -5.65e-05  | 4.7e-05  | -1.202 | 0.229 | -0.000    | 3.56e-05 |
| <b>p_poverty</b>  | 68.3897    | 34.410   | 1.987  | 0.047 | 0.946     | 135.833  |
| <b>p_hispanic</b> | 5.4010     | 7.142    | 0.756  | 0.449 | -8.596    | 19.398   |
| <b>p_black</b>    | 138.6586   | 30.014   | 4.620  | 0.000 | 79.831    | 197.486  |
| <b>Year</b>       | 1.0817     | 0.819    | 1.321  | 0.187 | -0.523    | 2.687    |

|                       |        |                          |          |
|-----------------------|--------|--------------------------|----------|
| <b>Omnibus:</b>       | 90.949 | <b>Durbin-Watson:</b>    | 0.605    |
| <b>Prob(Omnibus):</b> | 0.000  | <b>Jarque-Bera (JB):</b> | 160.066  |
| <b>Skew:</b>          | 1.305  | <b>Prob(JB):</b>         | 1.75e-35 |
| <b>Kurtosis:</b>      | 4.671  | <b>Cond. No.</b>         | 2.78e+08 |

Notes:

[1] Standard Errors are heteroscedasticity robust (HC3)

[2] The condition number is large, 2.78e+08. This might indicate that there are strong multicollinearity or other numerical problems.

## EDA for Asthma Data

In [82]: `asthma_sd[['Number_of_Asthma_ED_Visits', 'Age_Adjusted_Rate_of_Asthma_ED_V']].descr`

Out[82]:

|              | Number_of_Asthma_ED_Visits | Age_Adjusted_Rate_of_Asthma_ED_V |
|--------------|----------------------------|----------------------------------|
| <b>count</b> | 1510.000000                | 1510.000000                      |
| <b>mean</b>  | 65.754967                  | 35.670265                        |
| <b>std</b>   | 57.362169                  | 26.101186                        |
| <b>min</b>   | 12.000000                  | 3.700000                         |
| <b>25%</b>   | 26.000000                  | 17.000000                        |
| <b>50%</b>   | 46.000000                  | 28.900000                        |
| <b>75%</b>   | 83.000000                  | 44.600000                        |
| <b>max</b>   | 378.000000                 | 168.600000                       |

In [83]: `asthma_sd.head()`

Out[83]:

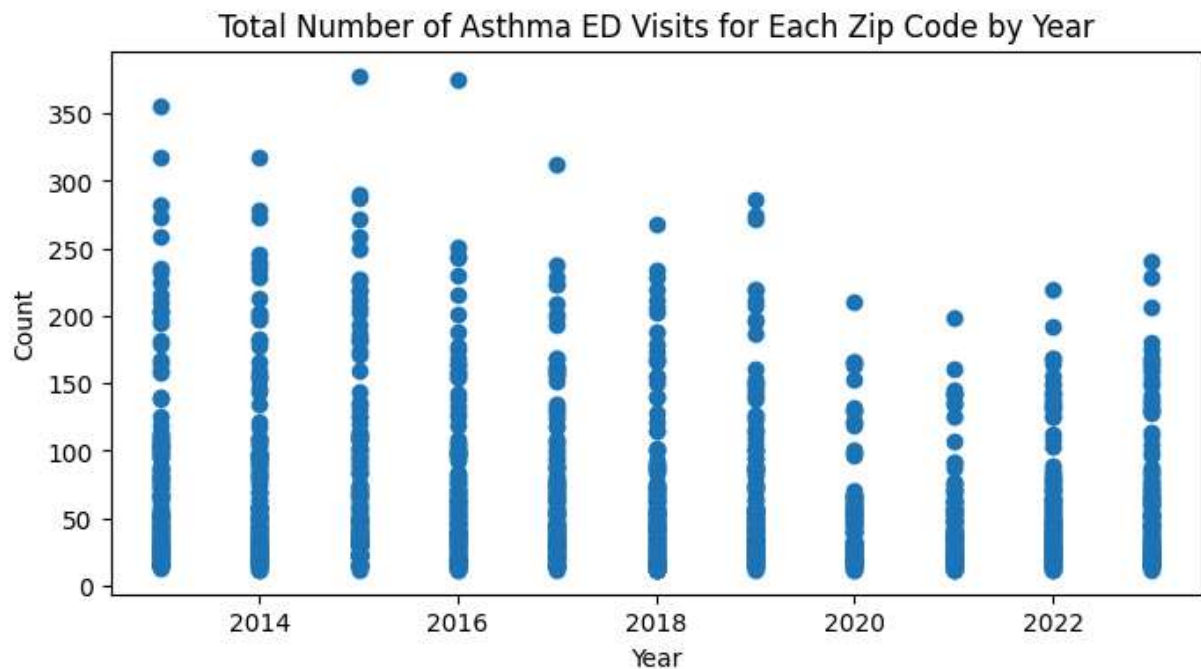
|            | _id | Year | Zip_Code | Number_of_Asthma_ED_Visits | Age_Adjusted_Rate_of_Asthma_ED_V |
|------------|-----|------|----------|----------------------------|----------------------------------|
| <b>244</b> | 245 | 2013 | 91902    | 23                         | 63.9                             |
| <b>245</b> | 246 | 2013 | 91910    | 159                        | 89.8                             |
| <b>246</b> | 247 | 2013 | 91911    | 210                        | 93.4                             |
| <b>247</b> | 248 | 2013 | 91913    | 51                         | 41.9                             |
| <b>248</b> | 249 | 2013 | 91915    | 28                         | 35.0                             |



```
In [84]: plt.scatter(asthma_sd['Year'], asthma_sd['Number_of_Asthma_ED_Visits'])
plt.xlabel('Year')
plt.ylabel('Count')
plt.title('Total Number of Asthma ED Visits for Each Zip Code by Year')
```

Out[84]: Text(0.5, 1.0, 'Total Number of Asthma ED Visits for Each Zip Code by Year')





## ANOVA Testing for AQI Between the 10 Most Populated Coastal and Inland Zip Codes

### Checking Conditions

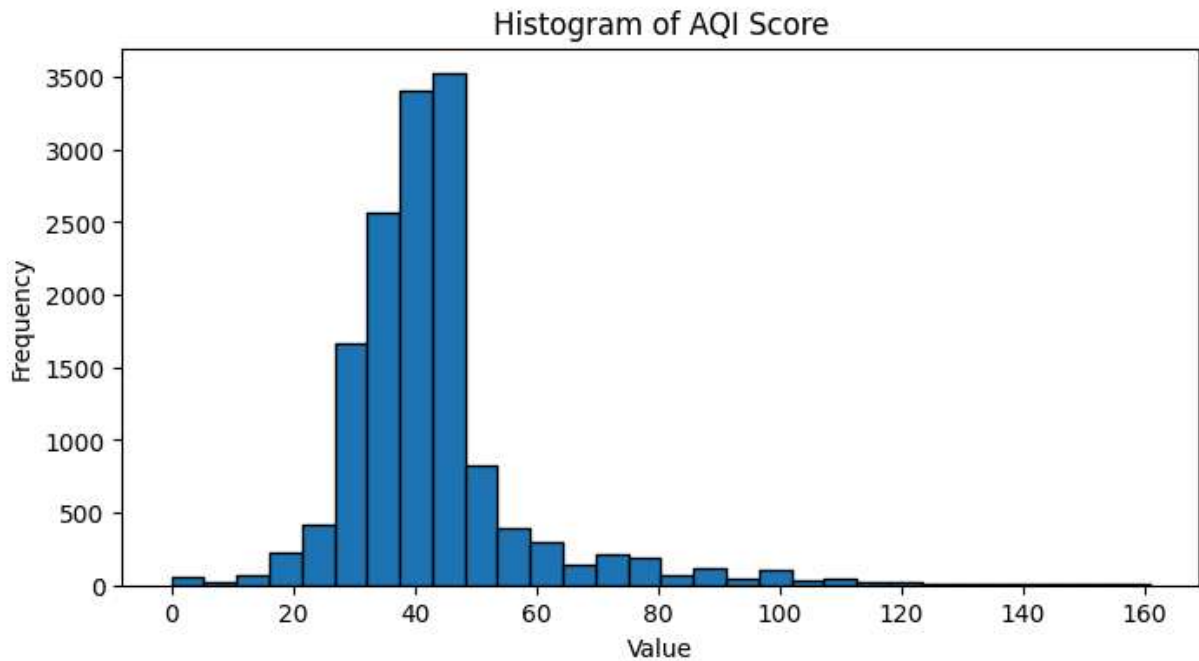
In order to do ANOVA testing, the data must fulfill 3 conditions.

First, each observation must be independent. This can be reasonably assumed to be fulfilled even though there are broader trends within the AQI score for each date, because our ANOVA testing does not discern based on date.

Next, the dependent variable must be normally distributed. We can check that with a histogram below. While the data has a slight right skew, it is not too far from having a Gaussian distribution, and thus this condition can be considered to be met.

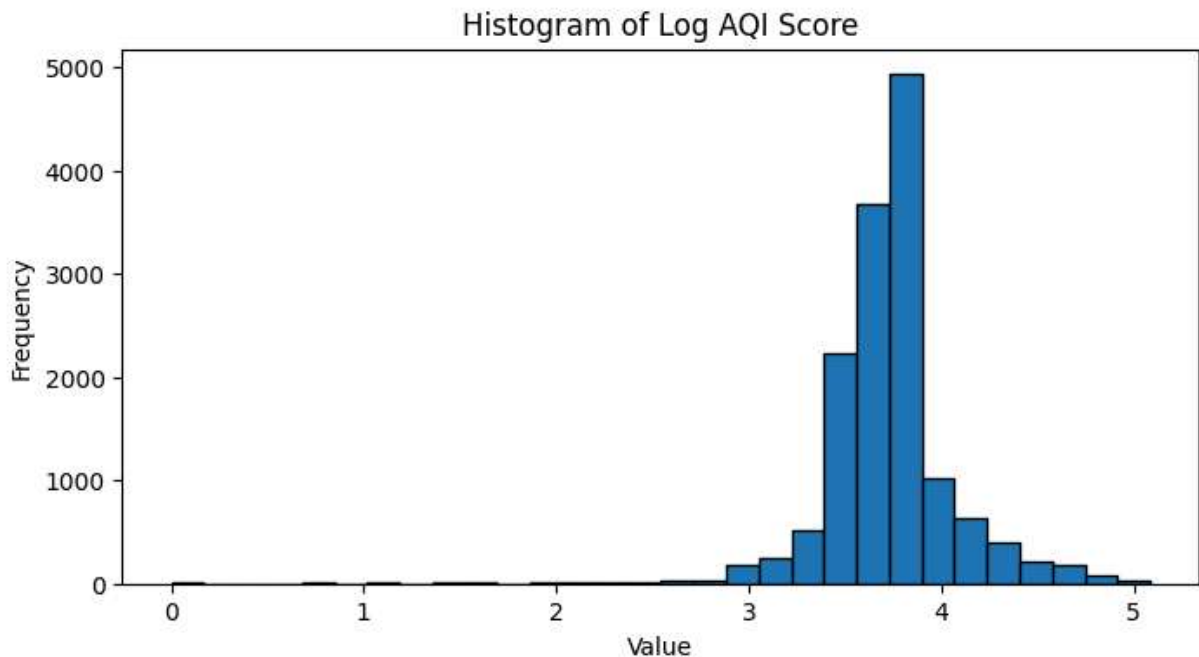
Third, the variability between each group must be similar. This can be checked with a box plot below. The variability for Petco Park and La Jolla appear to be higher than the other regions. In order to mitigate this, we should drop outliers.

```
In [85]: plt.hist(aqi_df['Daily AQI Value'], bins=30, edgecolor='black') # bins argument con
plt.title('Histogram of AQI Score')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```

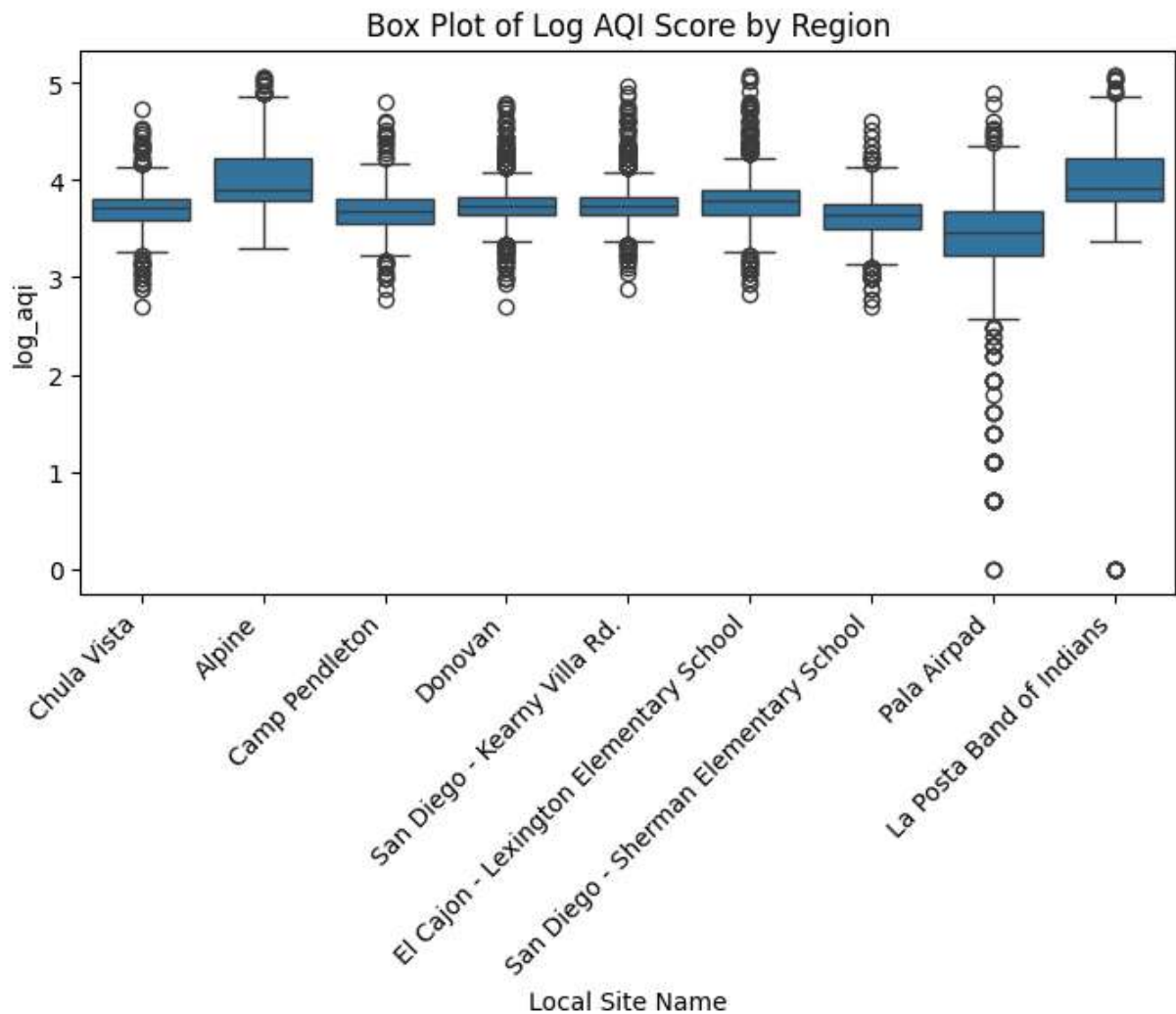


```
In [86]: aqi_df['log_aqi'] = np.log1p(aqi_df['Daily AQI Value'])

plt.hist(aqi_df['log_aqi'], bins=30, edgecolor='black') # bins argument controls th
plt.title('Histogram of Log AQI Score')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.show()
```



```
In [87]: sns.boxplot(x='Local Site Name', y='log_aqi', data=aqi_df.reset_index())
plt.title('Box Plot of Log AQI Score by Region')
plt.xticks(rotation=45, ha='right')
plt.show()
```



```
In [88]: regions = aqi_df['Local Site Name'].unique()
```

```
chula = np.array(aqi_df[aqi_df['Local Site Name']=='Chula Vista']['log_aqi'])
alpine = np.array(aqi_df[aqi_df['Local Site Name']=='Alpine']['log_aqi'])
pendleton = np.array(aqi_df[aqi_df['Local Site Name']=='Camp Pendleton']['log_aqi'])
donovan = np.array(aqi_df[aqi_df['Local Site Name']=='Donovan']['log_aqi'])
kearny = np.array(aqi_df[aqi_df['Local Site Name']=='San Diego - Kearny Villa Rd.'])
cajon = np.array(aqi_df[aqi_df['Local Site Name']=='El Cajon - Lexington Elementary'])
sherman = np.array(aqi_df[aqi_df['Local Site Name']=='San Diego - Sherman Elementary'])
pala = np.array(aqi_df[aqi_df['Local Site Name']=='Pala Airpad']['log_aqi'])
la_posta = np.array(aqi_df[aqi_df['Local Site Name']=='La Posta Band of Indians']['log_aqi'])
```

```
In [89]: f_statistic, p_value = stats.f_oneway(chula, alpine, pendleton, donovan, kearny, cajon, sherman, pala, la_posta)

print(f"F-statistic: {f_statistic}")
print(f"P-value: {p_value}")
```

```
F-statistic: 548.5690908436666
P-value: 0.0
```

```
In [90]: pivot = aqi_df.pivot_table(
    index='Date',
    columns='Local Site Name',
    values='Daily AQI Value')
```

```

)

coastal = pivot[coastal_stations].mean(axis=1)
inland = pivot[inland_stations].mean(axis=1)

coastal_smooth = coastal.rolling(window=7, center=True).mean()
inland_smooth = inland.rolling(window=7, center=True).mean()

plt.figure(figsize=(12,5))

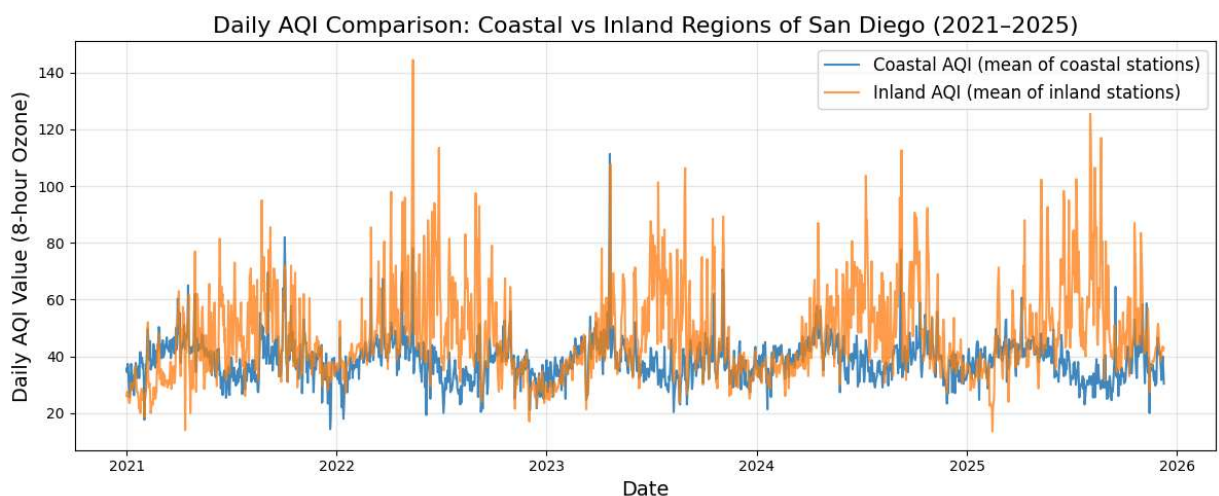
plt.plot(coastal, label="Coastal AQI (mean of coastal stations)", alpha=0.85)
plt.plot(inland, label="Inland AQI (mean of inland stations)", alpha=0.75)

plt.title("Daily AQI Comparison: Coastal vs Inland Regions of San Diego (2021-2025)")
plt.xlabel("Date", fontsize=14)
plt.ylabel("Daily AQI Value (8-hour Ozone)", fontsize=14)

plt.legend(fontsize=12)
plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()

```



## T-Test and Bootstrapping Analysis

Because the main question that we want to answer is if the distributions of AQI for coastal and inland regions, which are essentially two large samples, are different, we will do a 2 sample T-test, using Welch's correction, as the variances are different.

```

In [91]: ## Obtaining coastal and inland AQI distribution arrays
coastal = aqi_df[aqi_df['Coastal']==1]['Daily AQI Value']
inland = aqi_df[aqi_df['Inland']==1]['Daily AQI Value']

## 2 Sample T-Test
result = pg.ttest(coastal, inland, correction=True)
print(result)

```

|        | T         | dof         | alternative | p-val         | CI95%         | \ |
|--------|-----------|-------------|-------------|---------------|---------------|---|
| T-test | -24.86141 | 5287.111907 | two-sided   | 3.945696e-129 | [-9.72, -8.3] |   |

|        | cohen-d  | BF10       | power |
|--------|----------|------------|-------|
| T-test | 0.543345 | 1.493e+128 | 1.0   |

```
In [92]: ## sampling from inland n times and coastal n times, both with replacement.
## Then you plot a histogram for both and you should get a bell curve for both.
## Then you layer one over the other and one should be shifted compared to the other
iters = 1000
coastal_boot = []
for _ in range(iters):
    bootstrap_sample = np.random.choice(coastal, size=len(coastal), replace=True)
    coastal_boot.append(bootstrap_sample.mean())

inland_boot = []
for _ in range(iters):
    bootstrap_sample = np.random.choice(inland, size=len(inland), replace=True)
    inland_boot.append(bootstrap_sample.mean())
```

```
In [93]: plt.hist(coastal_boot, bins=30, alpha=0.5, label='Coastal', color='blue')
plt.hist(inland_boot, bins=30, alpha=0.5, label='Inland', color='red')
plt.title('Distribution of Mean Daily AQI for Coastal and Inland Regions')
plt.legend()
```

Out[93]: <matplotlib.legend.Legend at 0x181d2ab4460>

